MODULE 5

Java Collections Framework

Store, manipulate, and organize data efficiently with the interfaces and classes of java.util.

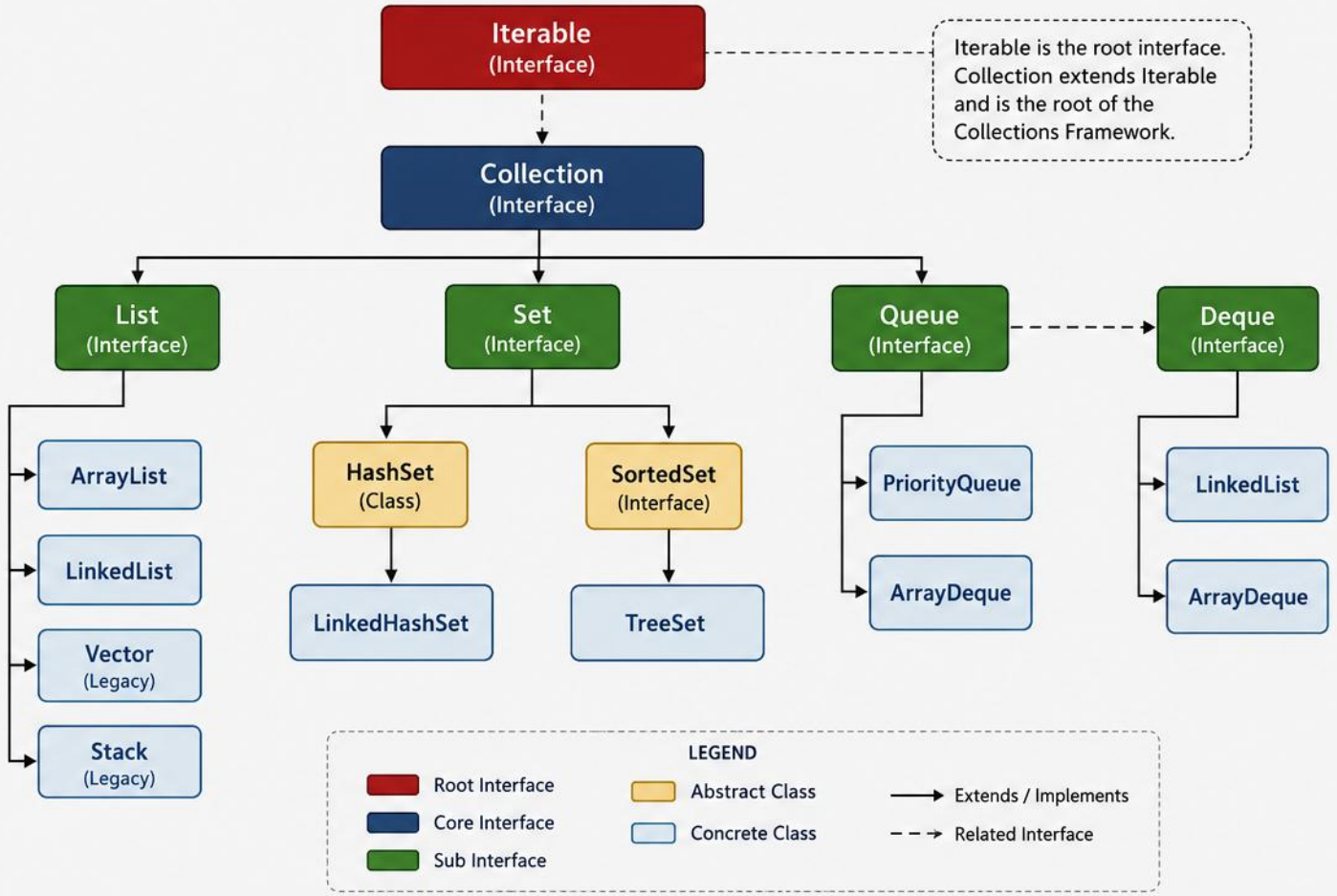
TEKsystems Global Services

WORKFORCE DEVELOPMENT

TEKsystems Global Services

WORKFORCE DEVELOPMENT

JAVA COLLECTIONS FRAMEWORK



COLLECTIONS FRAMEWORK KEY INTERFACES

Iterable
The root interface that allows foreach iteration.

Collection
The root interface for all collection types.

List
An ordered collection (may contain duplicates).

Set
A collection that does not allow duplicates.

Queue
A collection designed for holding elements prior to processing.

Deque
A double-ended queue that supports adding/removing elements from both ends.

COLLECTIONS FRAMEWORK PROVIDES

- High Performance and Efficiency
- Standardized Architecture
- Interoperability and Reusability
- Scalability and Flexibility
- Type Safety and Generics Support

MODULE OVERVIEW

The Java Collections Framework provides interfaces and classes to store, manipulate, and organize data efficiently.

LEARNING OBJECTIVES

- Understand the architecture of the Collections Framework.
- Work with Lists, Sets, Queues, and Maps.
- Choose the right collection for a given use case.
- Apply common operations and iterate effectively.
- Use utility classes and best practices.

⭐ Framework Features

- Reusable — pre-built structures & algorithms
- Efficient — optimized for performance
- Type Safe — Generics for compile-time safety

🤔 When to Use What?

- List: ordered, duplicates allowed
- Set: unique elements, no duplicates
- Queue: FIFO / priority processing
- Map: key–value pairs

KEY TAKEAWAY Think: what am I storing, do I need order, duplicates, or fast lookups? That guides your choice.

COLLECTIONS IN REAL-WORLD APPLICATIONS

Five domains where collections do the heavy lifting every day.

E-Commerce

Store products, manage carts, track orders and history.

Banking

Manage accounts, store transactions, detect fraud patterns.

Social Media

Store users/friends, maintain feeds, manage likes and comments.

Analytics & Reporting

Aggregate data, group and summarize, produce fast reports.

Enterprise Systems

Cache data, manage configuration, handle job queues.

KEY TAKEAWAY Wherever an application needs to store more than one thing, collections are doing the work — that's the efficient data handling this section is really about.

THE ALTERNATIVE — WITHOUT COLLECTIONS

Manual array management is verbose, error-prone, and slow to write.

WITHOUT COLLECTIONS

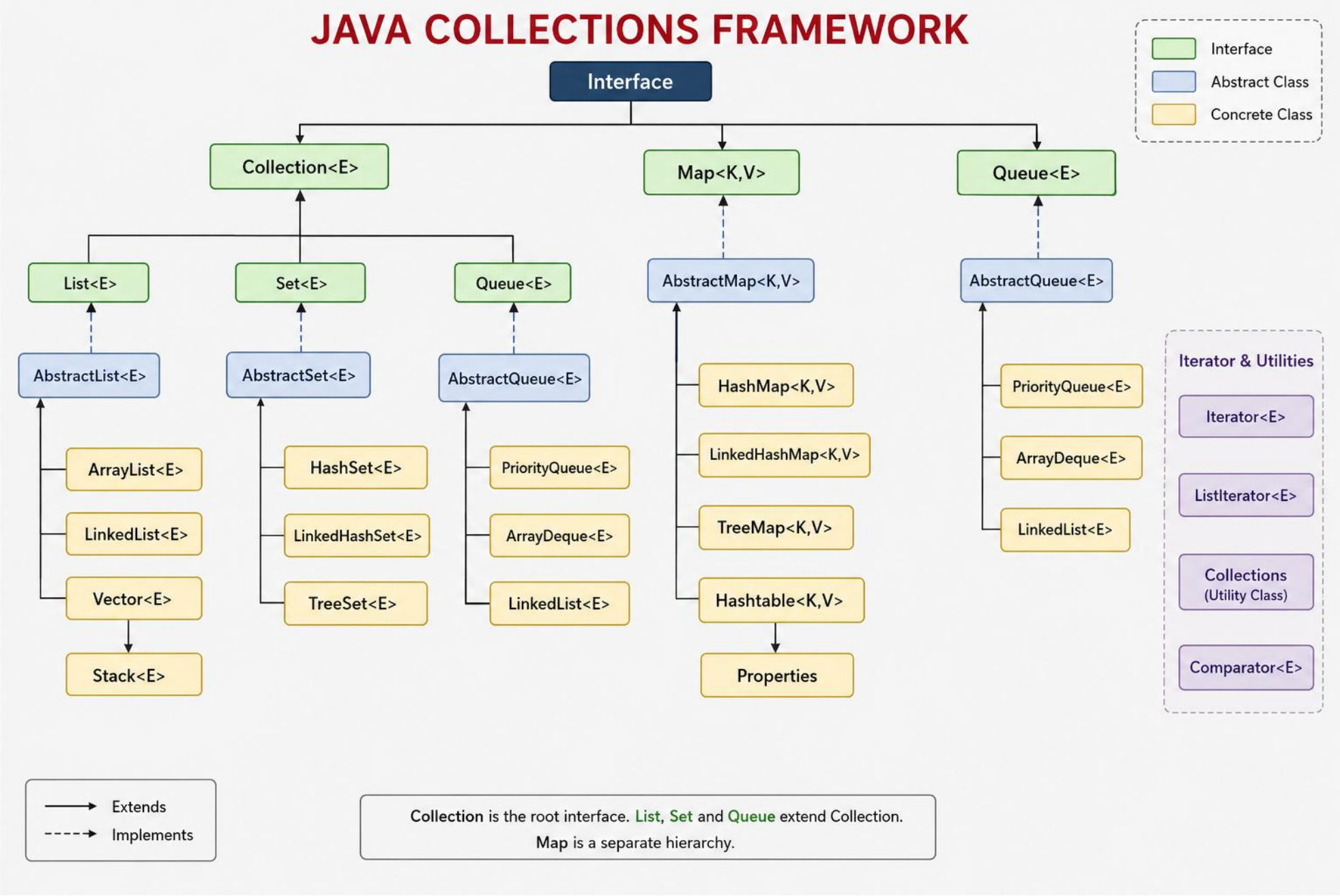
```
String[] users = new String[10];
int size = 0;
if (size == users.length) {
  String[] newArr = new String[users.length * 2];
  System.arraycopy(users, 0, newArr, 0, size);
  users = newArr;
}
users[size++] = "Alice";  // lots of manual work!
```

WITH COLLECTIONS

```
List<String> users = new ArrayList<>();
users.add("Alice");
users.add("Bob");
users.add("Charlie");
for (String user : users) {
    System.out.println(user);
}
// Simple, clean, and efficient!
```

- Without: manual resizing, more code and complexity, more bugs, poor performance.
- With: no manual resizing, less code, fewer bugs, better performance, easy to scale.

KEY TAKEAWAY Collections let you focus on business logic instead of low-level data management.



LIST INTERFACE

List<E> (Interface)

An ordered collection (also called sequence).
Allows duplicate elements and maintains insertion order.

extends

ArrayList<E>

Resizable-array implementation of List.

- Fast random access
- Slow insert/delete in middle

LinkedList<E>

Doubly-linked list implementation of List.

- Fast insert/delete
- Slow random access

Vector<E>

Legacy class, synchronized. Similar to ArrayList but thread-safe.
(Use ArrayList instead)

Stack<E>

Legacy class that extends Vector. Represents LIFO stack.
(Use Deque instead)

Key Characteristics

- Ordered collection (maintains insertion order)
- Allows duplicate elements
- Elements are accessed by index (0 to size-1)
- Part of `java.util` package
- Extends `Collection<E>` interface

Common Methods

boolean add(E e)	Appends element to the end
void add(int index, E element)	Inserts element at specified position
E get(int index)	Returns element at specified index
E set(int index, E element)	Replaces element at specified index
E remove(int index)	Removes element at specified index
int indexOf(Object o)	Returns index of first occurrence
int size()	Returns number of elements
boolean isEmpty()	Checks if list is empty

List Example

```
List<String> names = new ArrayList<>();
names.add("Alice");
names.add("Bob");
names.add("Charlie");

// Access by index
System.out.println(names.get(1);   // Bob

// Iterate
for (String name : names) {
    System.out.println(name);
}
```

When to Use Which Implementation?

Use **ArrayList** when:

- Frequent access by index
- Few insertions/deletions in middle

Use **LinkedList** when:

- Frequent insertions/deletions
- Less random access

Avoid Vector & Stack:

- Legacy classes
- Use modern alternatives

General Recommendation:

- Prefer List<E> over specific implementations in code

CHOOSING A LIST IMPLEMENTATION

How ArrayList, LinkedList, and Vector differ — and when each wins.

IMPLEMENTATION	STRUCTURE	USE WHEN…
`ArrayList`	Resizable Array	More reads than writes; random access needed
`LinkedList`	Doubly-Linked List	Frequent additions/removals in the middle
`Vector`	Resizable Array (Synchronized)	Need thread safety or maintaining legacy code

- Zero-Based Indexing: first element is at index 0.
- Duplicates and multiple null values are allowed.
- Order is maintained in insertion order.
- ArrayList provides O(1) get()/set(); LinkedList provides O(n).
- Choosing: ArrayList for indexed reads/append-heavy work; LinkedList for front/end inserts (also a Deque); for arbitrary-index inserts, neither wins automatically — measure.
- List in general: ordered, index-based access, duplicates allowed.

KEY TAKEAWAY Random-access performance is the single biggest differentiator between ArrayList and LinkedList.

EXAMPLE: USING LIST

Add, access, update, remove, and iterate — the five core operations.

LIST METHODS
- add(e) / add(i, e)
- get(i) / set(i, e)
- remove(i) / indexOf(o)
- size() / isEmpty()

```
List<String> list = new ArrayList<>();
list.add("Apple");                    // [Apple]
list.add("Banana");                   // [Apple, Banana]
list.add(1, "Blueberry");             // [Apple, Blueberry, Banana]

System.out.println(list.get(2));      // Banana
list.set(2, "Blackberry");             // updates index 2
list.remove("Apple");                  // [Blueberry, Blackberry]

for (String fruit : list) {
    System.out.print(fruit + " ");
}
```

KEY TAKEAWAY add(index, e) shifts everything after it — that's the O(n) cost hiding behind a simple insert.

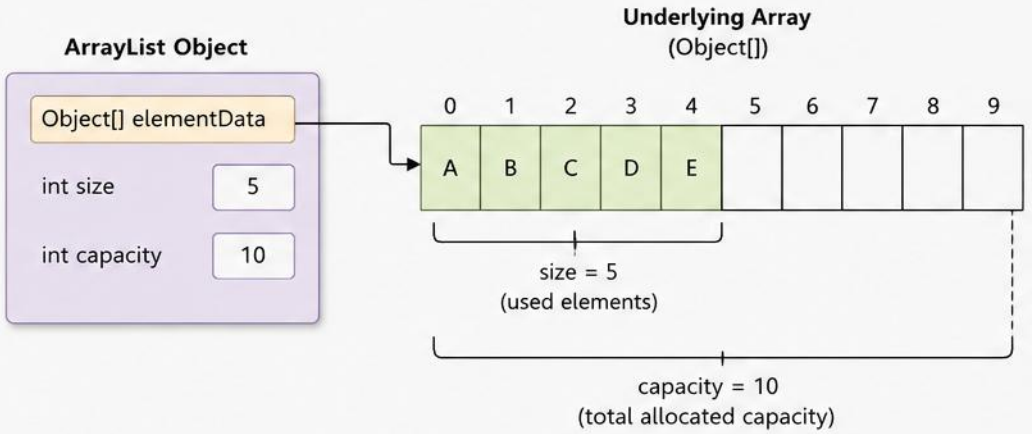
ARRAYLIST (java.util.ArrayList)

ArrayList is a resizable array implementation of the List interface.
It allows dynamic storage of elements with random access.

KEY FEATURES

- Resizes automatically when capacity is full
- Maintains insertion order
- Allows duplicate elements
- Allows null elements
- Provides fast random access using index
- Not synchronized (not thread-safe)

INTERNAL STRUCTURE



COMMON OPERATIONS

`add(E e)`
Adds element at the end

`add(int index, E e)`
Inserts element at specified index

`get(int index)`
Returns element at specified index

`set(int index, E e)`
Replaces element at specified index

`remove(int index)`
Removes element at specified index

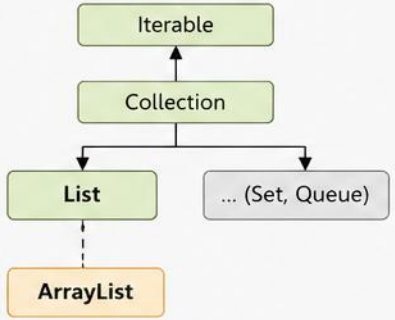
`remove(Object o)`
Removes first occurrence of object

`size()`
Returns number of elements

`isEmpty()`
Checks if list is empty

`clear()`
Removes all elements

COLLECTION FRAMEWORK HIERARCHY



EXAMPLE

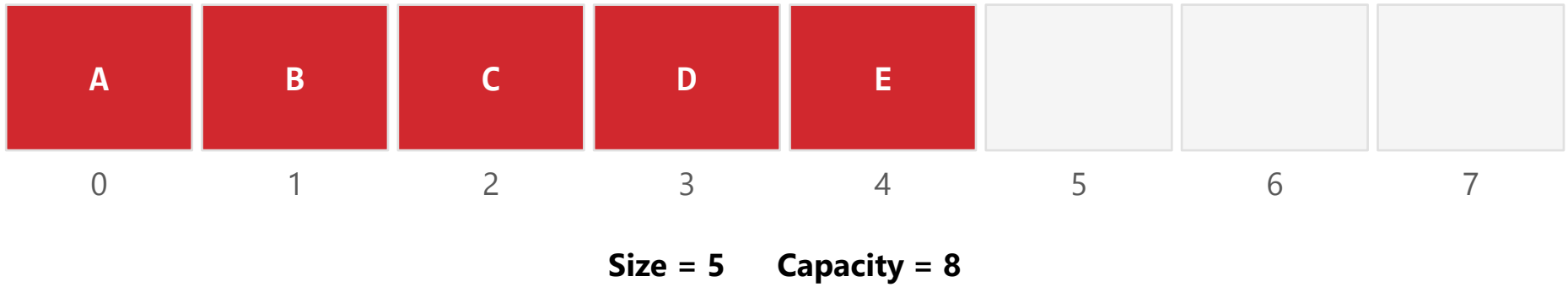
```
ArrayList<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");
list.add("D");
list.add("E");

// Internal State
// elementData = ["A", "B", "C", "D", "E", null, null, null, null, null]
// size = 5, capacity = 10
```

KEY POINTS

- Fast random access O(1) using index
- Add at end amortized O(1)
- Insert/Delete in middle O(n) (elements shifted)
- Resizes automatically (grows by 1.5x)

ARRAYLIST — UNDERLYING STRUCTURE

A resizable object array; size tracks elements used, capacity tracks total slots.



HOW IT GROWS

When size exceeds capacity, a new array with a larger capacity is created (typically about 1.5× the previous capacity), and all elements are copied over.

KEY TAKEAWAY Growth is amortized O(1) — occasional resizes are expensive, but rare enough that the average stays cheap.

ARRAYLIST — PERFORMANCE CHARACTERISTICS

Index-based operations are fast; anything requiring a shift is not.

OPERATION	COMPLEXITY	EXPLANATION
`get(index) / set(index, e)`	O(1)	Direct access / update by index
`add(e) at end`	Amortized O(1)	Usually O(1); occasionally O(n) during resize
`add(index, e)`	O(n)	Elements after index are shifted
`remove(index)`	O(n)	Elements after index are shifted
`contains(Object)`	O(n)	Linear search

KEY TAKEAWAY ArrayList trades slow middle-insertion for fast everything-else — that's the core design trade-off. Pre-size with ensureCapacity() to avoid resize/copy costs.

EXAMPLE: BASIC USAGE

Add, insert, access, and remove — with output shown inline.

```
ArrayList<String> list = new ArrayList<>();
list.add("Apple");
list.add("Banana");
list.add("Cherry");
list.add(1, "Blueberry");          // insert at index 1

System.out.println(list.get(2));   // Banana
System.out.println(list);          // [Apple, Blueberry, Banana, Cherry]
list.remove("Banana");
System.out.println(list);          // [Apple, Blueberry, Cherry]
```

KEY TAKEAWAY Insert-at-index shifts every following element — watch out for this in performance-sensitive loops.

TRY IT YOURSELF — EXERCISE 1: WORKING WITH LIST

Reinforces: ArrayList, as just covered.

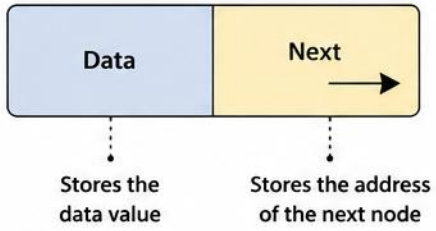
#	STEP	CODE	RESULT
1	Goal	Use ArrayList<String> — add, update, remove, search, and iterate	Ordered list, duplicates allowed
2	Declare	List<String> books = new ArrayList<>();	Empty, ordered list ready
3	Populate	books.add("Dune"); books.add("Dune");	Duplicates are kept, unlike a Set
4	Operate	books.set(0, "Dune (2nd ed.)"); books.remove(1);	Update by index, remove by index
5	Iterate	for (String b : books) System.out.println(b);	Prints in insertion order
6	Key concept	List preserves insertion order and allows index-based access	Ties to the List interface you just saw
7	Reference	exercises/exercise-01-arraylist.md	Full starter code and steps

TRY IT NOW Complete Exercise 1 before continuing — see exercises/exercise-01-arraylist.md.

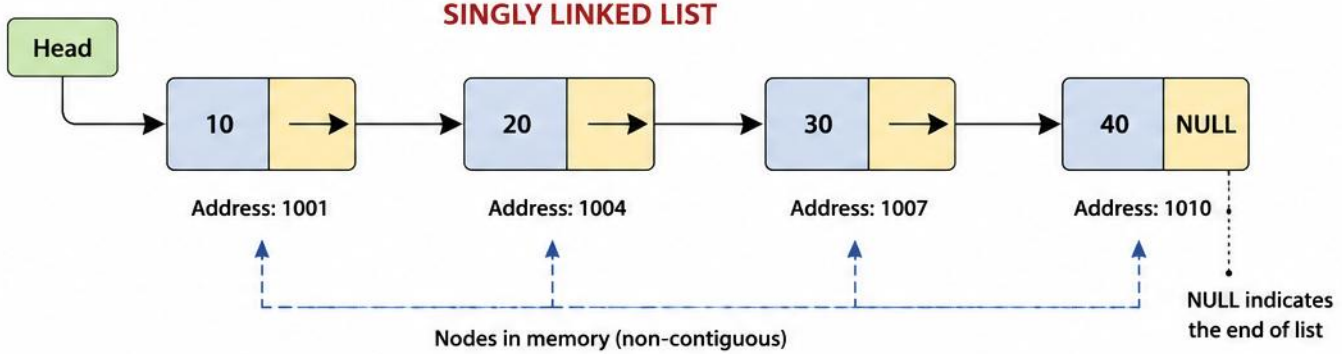
LINKED LIST

A Linked List is a linear data structure where elements (nodes) are stored in non-contiguous memory locations.
Each node contains data and a reference (link) to the next node in the sequence.

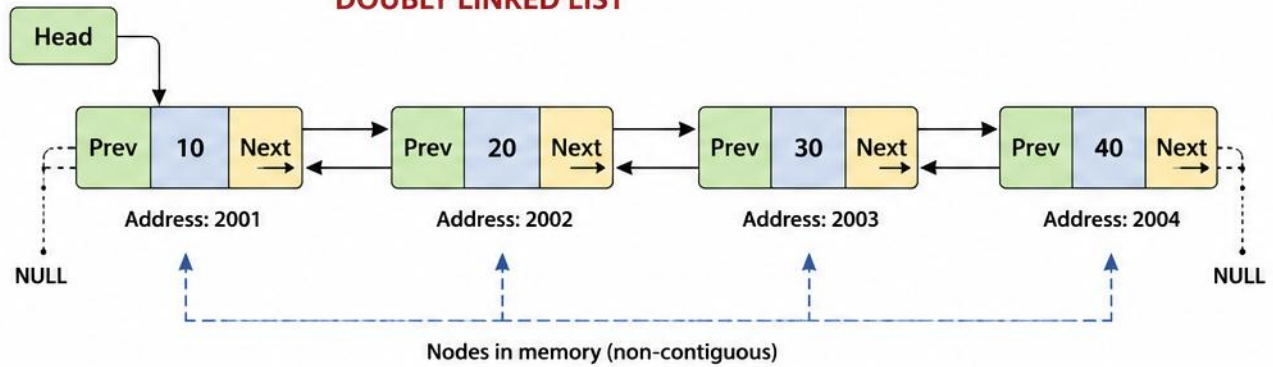
NODE STRUCTURE



SINGLY LINKED LIST



DOUBLY LINKED LIST



- Prev – Address of previous node
- Data – Actual data
- Next – Address of next node

COMMON OPERATIONS

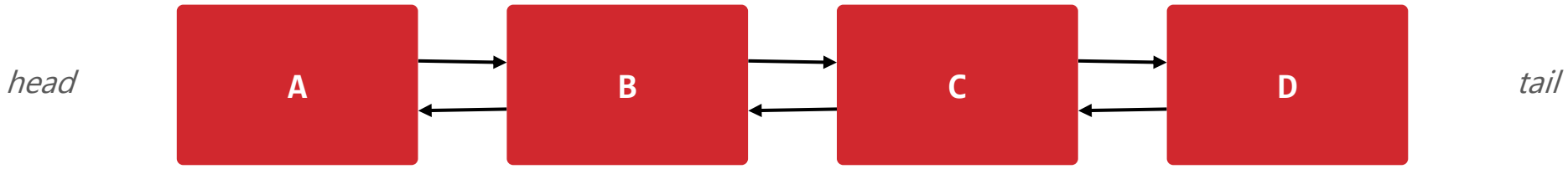
Insertion	Add a new node at the beginning, end or any position.
Deletion	Remove a node from the beginning, end or any position.
Traversal	Visit each node in the list sequentially.
Search	Find a node with a specific value.
Update	Modify the data of an existing node.

ADVANTAGES

- Dynamic size
- Efficient insertion and deletion
- Does not require contiguous memory

LINKEDLIST — UNDERLYING STRUCTURE

Each node stores data plus references to the previous and next nodes.



HOW IT WORKS

- Each node stores the element (data), a reference to the previous node, and a reference to the next node.
- Insertion/deletion just relinks pointers — no shifting of other elements required.

KEY TAKEAWAY Because nodes only relink pointers, insert/delete anywhere is O(1) once you've located the position.

LINKEDLIST — PERFORMANCE CHARACTERISTICS

Fast at the ends, slow in the middle — the opposite of ArrayList.

OPERATION	COMPLEXITY	EXPLANATION
`get(index) / set(index, e)`	O(n)	Must traverse from head or tail
`add(e) at end`	O(1)	Add at tail in constant time
`add(index, e) / remove(index)`	O(n)	Traverse to index, then insert/remove
`addFirst() / addLast()`	O(1)	Constant time
`removeFirst() / removeLast()`	O(1)	Constant time

KEY TAKEAWAY Traversal, not the operation itself, is what makes LinkedList's random access O(n). Insert itself is O(1) once positioned, but reaching that position is still O(n).

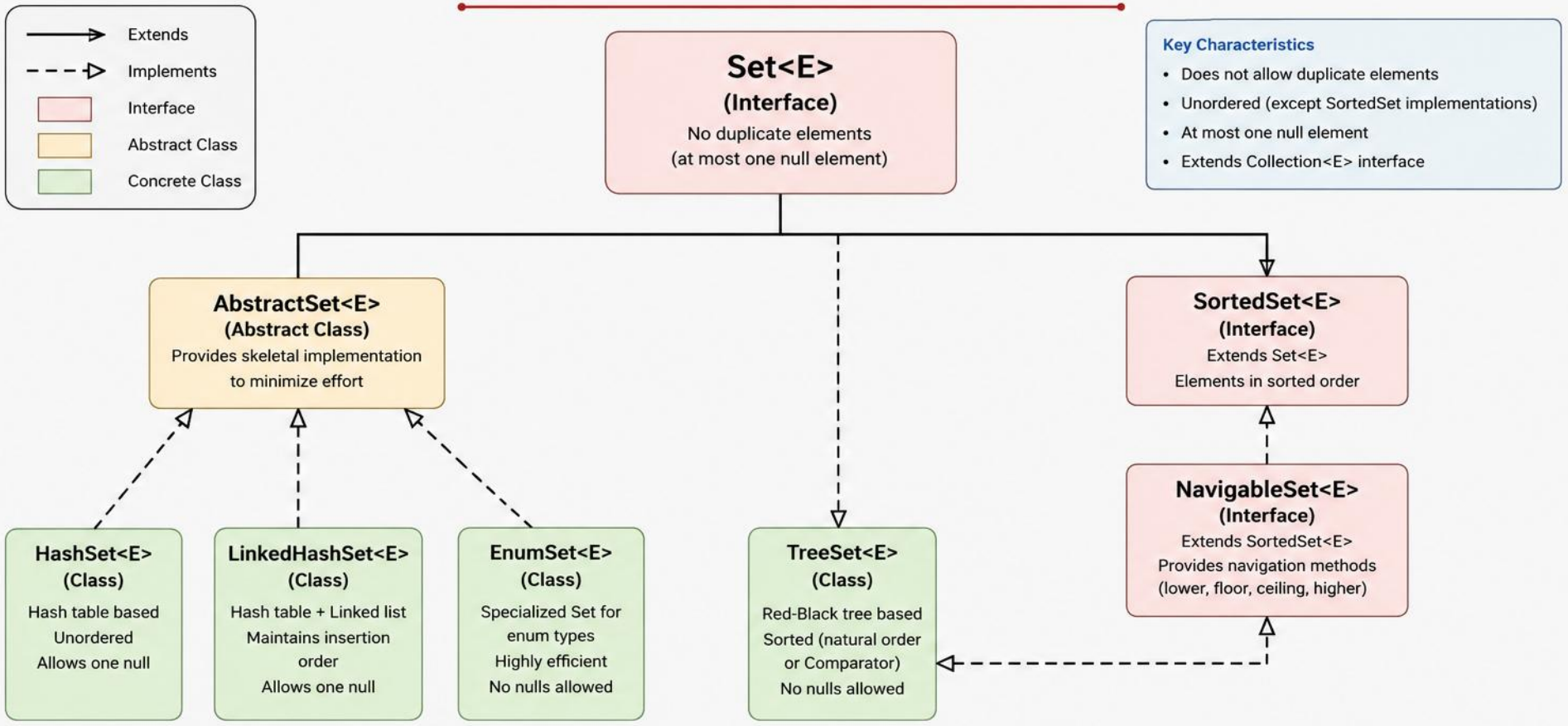
EXAMPLE: BASIC USAGE

addFirst/addLast, indexed insert, and getFirst/getLast in action.

```
LinkedList<String> list = new LinkedList<>();
list.add("Apple"); list.add("Banana"); list.add("Cherry");
list.addFirst("Start");   list.addLast("End");
list.add(2, "Blueberry");

System.out.println("First: " + list.getFirst());  // Start
System.out.println("Last: " + list.getLast());    // End

list.remove("Apple"); list.removeFirst(); list.removeLast();
// Remaining: Blueberry Banana Cherry
```

KEY TAKEAWAY getFirst()/getLast() and addFirst()/addLast() are constant time — LinkedList is a great Deque.

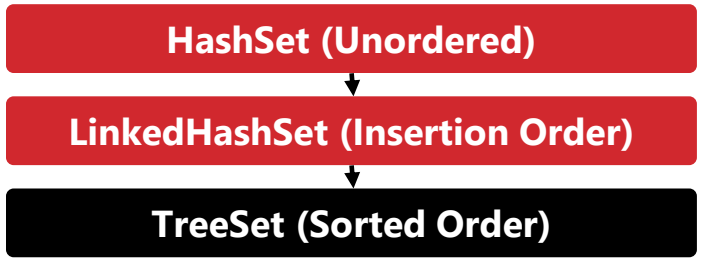
SET INTERFACE IN JAVA



SET — DECLARATION & HIERARCHY

No additional methods beyond Collection — all behavior comes from the Set contract.

```
public interface Set<E> extends Collection<E> {
    // no additional methods
}
```



CHARACTERISTIC	BEHAVIOR
Uniqueness	No duplicate elements (based on equals()).
Null Handling	At most one null allowed (except TreeSet).
Equality	Two sets are equal if they contain the same elements.
Indexed Access	Not supported — no get(index) or set(index).

KEY TAKEAWAY Set uses equals() and hashCode() to determine uniqueness — indexed access simply doesn't exist.

SET — METHODS & IMPLEMENTATIONS

Every method is inherited from Collection; the three implementations differ only in ordering.

METHOD	DESCRIPTION
`add(E e)`	Adds element if not already present
`remove(Object o) / contains(Object o)`	Removes / checks for the element
`size() / isEmpty() / clear()`	Count, empty check, and remove all

IMPLEMENTATION	ORDERING	PERFORMANCE	BEST FOR
`HashSet`	No order guaranteed	O(1) avg	Fast operations, order doesn't matter
`LinkedHashSet`	Insertion order	O(1) avg	When insertion order must be preserved
`TreeSet`	Sorted order	O(log n)	When sorted order / range ops are needed

KEY TAKEAWAY Set has no methods of its own — the contract is entirely about what add() and iteration guarantee. Use List instead if duplicates are needed.

EXAMPLE: USING SET

Duplicates are silently ignored; one null is allowed.

```
Set<String> set = new HashSet<>();
set.add("Apple");
set.add("Banana");
set.add("Apple");     // duplicate, ignored
set.add(null);        // one null allowed
set.add("Cherry");

System.out.println(set.contains("Banana"));  // true
set.remove("Apple");
System.out.println(set);  // [null, Banana, Cherry] (order may vary)
```

KEY TAKEAWAY HashSet's iteration order is not guaranteed and can change after resizing — never rely on it.

HASHSET

Implementation of Set using a hash table — unique elements, no guaranteed order, very fast.

Key Points

- Stores unique elements only.
- Allows one null element.
- No guaranteed order of elements.
- Uses a hash table for storage.
- Very fast add/remove/contains.

Constructors

- HashSet() — capacity 16, load factor 0.75
- HashSet(Collection<? extends E> c)
- HashSet(int initialCapacity)
- HashSet(int cap, float loadFactor)

KEY TAKEAWAY HashSet is the go-to implementation when you need uniqueness and speed, and order doesn't matter.

HASHSET

HashSet in Java is implemented using a HashMap.
It stores unique elements and does not maintain insertion order.

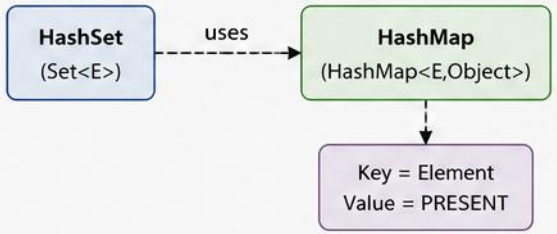
KEY CHARACTERISTICS

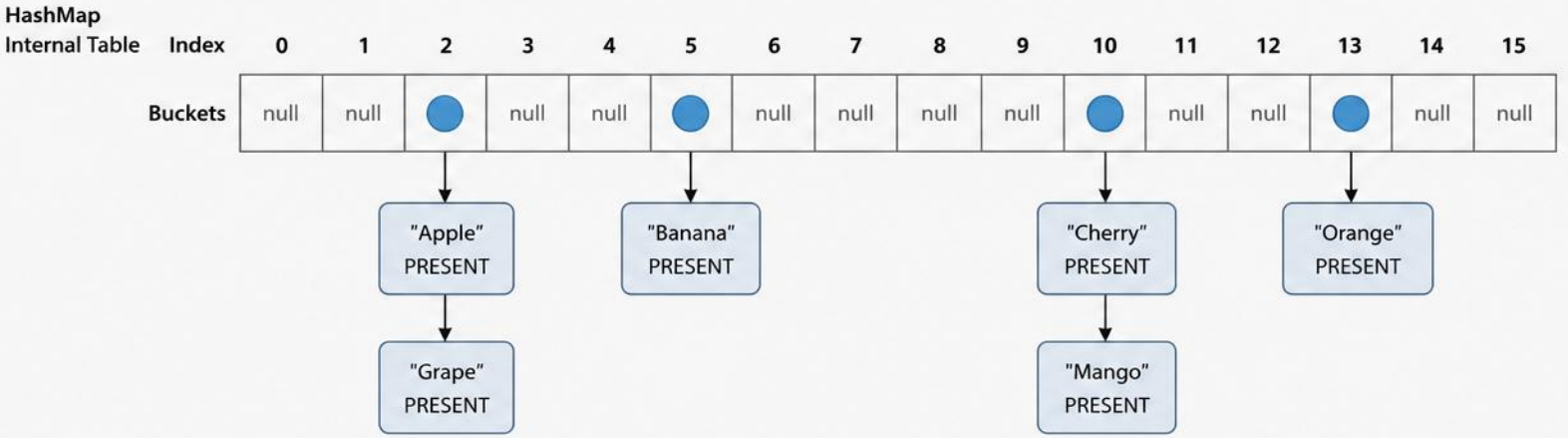
- Stores unique elements only
- Allows one null element
- Unordered (no insertion order)
- O(1) average time complexity for add, remove, contains
- Not synchronized (use Collections.synchronizedSet() for thread-safety)

Usage

```
Set<String> set = new HashSet<>();
set.add("Apple");
set.add("Banana");
set.add("Cherry");
set.add("Apple"); // Duplicate - not added
```

INTERNAL IMPLEMENTATION





HOW IT WORKS

1. Element is hashed to an index in the table.
2. If bucket is empty, element is added.
3. If bucket is not empty, element is added to the linked list (chaining).
4. Duplicate elements are not added.

EXAMPLE

- Add: Apple, Banana, Cherry, Grape, Mango, Orange, Apple
- Internal storage (HashMap): Keys = unique elements, Value = PRESENT
- Duplicate "Apple" is ignored.

NOTE

Since HashSet is backed by HashMap,
it does not guarantee any order of elements.
Use LinkedHashSet to maintain insertion order.
Use TreeSet to keep elements sorted.

LEGEND

- Bucket with at least one element
- Node (Key = Element, Value = PRESENT)
- Empty Bucket (null)

HOW HASHSET WORKS

Hashing determines the bucket; buckets absorb collisions via chaining (or trees in Java 8+).

- HashSet uses hashing to determine the bucket index for each element.
- Elements with the same hash code are stored in the same bucket (chaining).

DEFAULT CAPACITY	LOAD FACTOR	RESIZE TRIGGER	RESIZE RESULT
16	0.75	size > capacity × load factor	Capacity typically doubles

Always override both equals() and hashCode() in custom classes used in a HashSet.

KEY TAKEAWAY Two objects are duplicates only if equals() returns true AND hashCode() matches — both must be correct.

EXAMPLE: BASIC USAGE

Duplicates and repeated nulls are silently ignored.

```
Set<String> set = new HashSet<>();
set.add("Apple"); set.add("Banana"); set.add("Cherry");
set.add("Apple");    // duplicate (ignored)
set.add(null); set.add(null);   // only one null kept

System.out.println(set.contains("Banana"));  // true
System.out.println(set.remove("Apple"));      // true
System.out.println(set);   // [Banana, null, Cherry] (order may vary)
```

KEY TAKEAWAY When to use HashSet: eliminating duplicates, fast lookup, order doesn't matter. Core ops run O(1) average.

CUSTOM OBJECTS & SET OPERATIONS

Correct equals()/hashCode() are required for custom keys; union/intersection/difference use standard methods.

```
class Student {
  int id; String name;
  @Override
  public boolean equals(Object o) {
    if (!(o instanceof Student)) return false;
    return id == ((Student) o).id;
  }
  @Override
  public int hashCode() {
    return Objects.hash(id);
  }
}
```

```
Set<String> a = Set.of("A","B","C");
Set<String> b = Set.of("B","C","D");

// Union
union.addAll(b);          // [A,B,C,D]
// Intersection
intersection.retainAll(b);// [B,C]
// Difference
difference.removeAll(b);  // [A]
```

KEY TAKEAWAY retainAll(), addAll(), and removeAll() turn HashSet into a ready-made set-algebra toolkit — same O(1) average applies to custom objects.

TRY IT YOURSELF — EXERCISE 2: WORKING WITH SET

Reinforces: HashSet, as just covered.

#	STEP	CODE	RESULT
1	Goal	Add duplicate strings to a HashSet and prove uniqueness	Duplicates collapse to one entry
2	Declare	Set<String> tags = new HashSet<>();	Empty, unordered set ready
3	Populate	tags.add("java"); tags.add("java");	Second add is silently ignored
4	Verify	System.out.println(tags.size());	Prints 1, not 2
5	Optional	new TreeSet<>(tags)	Same elements, now in sorted order
6	Key concept	A Set enforces uniqueness — no duplicate elements, unlike a List	Ties to the Set interface you just saw
7	Reference	exercises/exercise-02-hashset.md	Full starter code and steps

TRY IT NOW Complete Exercise 2 before continuing — see exercises/exercise-02-hashset.md.

TREESET

Implementation of SortedSet using a Red-Black Tree — unique elements, always sorted.

Key Points

- Implements SortedSet<E> (extends Set<E>).
- Elements sorted ascending (natural or Comparator).
- No null element allowed.
- Guarantees O(log n) for basic operations.

Constructors

- TreeSet() — natural ordering
- TreeSet(Comparator<? super E> c)
- TreeSet(Collection<? extends E> c)
- TreeSet(SortedSet<E> s)

KEY TAKEAWAY TreeSet is the right choice when you need a set of unique elements that must always remain sorted.

TreeSet in Java

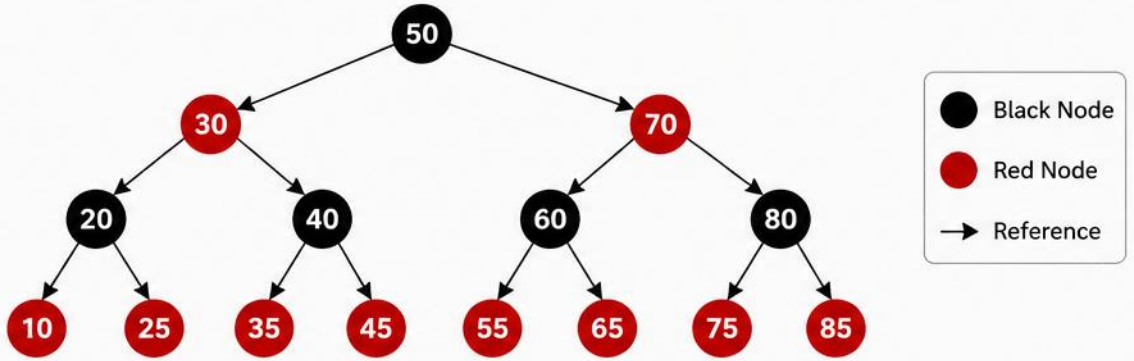
TreeSet is a class that implements the **NavigableSet** interface.
It stores elements in a **sorted order** (ascending by default) using a Red-Black Tree.

Key Features

- Stores unique elements only (no duplicates).
- Elements are sorted in natural ordering or by a custom **Comparator**.
- Provides efficient operations like adding, removing and retrieval.
- Implements **NavigableSet** interface.

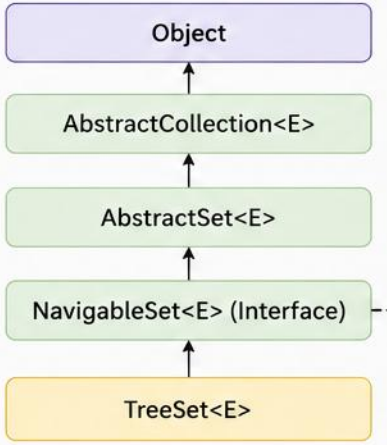
Internal Data Structure

Red-Black Tree



In-order traversal of the above tree: 10, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85
(Sorted Order)

Class Hierarchy



Provides Navigational Operations

- lower(), floor()
- ceiling(), higher()
- pollFirst(), pollLast()
- descendingSet()
- headSet(), tailSet()
- subSet()

TreeSet does not allow null elements.
It is not synchronized. Use Collections.synchronizedSortedSet() for thread safety.

Common Operations

Operation	Time Complexity
add(E e)	O(log n)
remove(Object o)	O(log n)
contains(Object o)	O(log n)
first() / last()	O(log n)
size()	O(1)
iterator()	O(n)

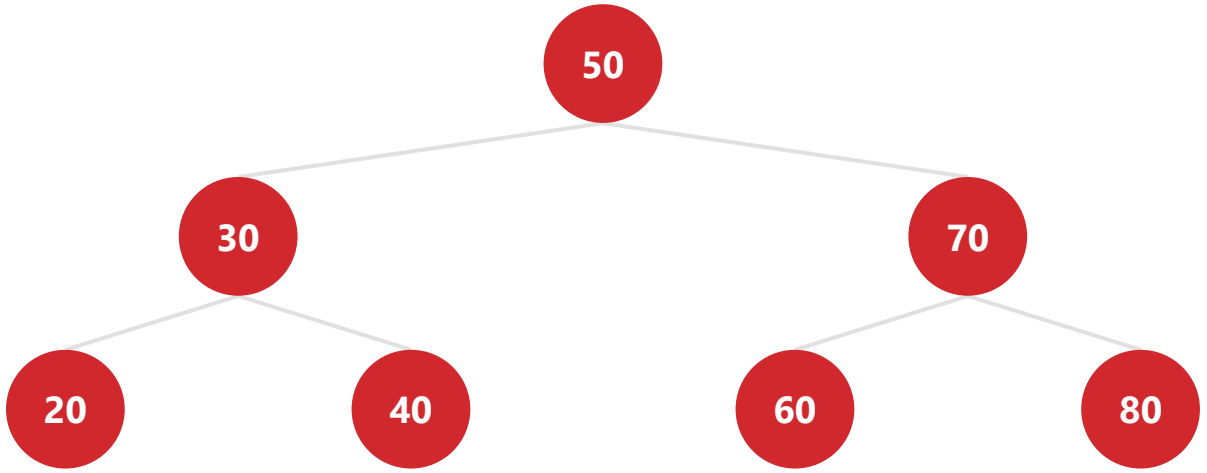
Ordering

- By default, elements are sorted in natural ascending order.
- To use custom order, provide a **Comparator** while creating **TreeSet**.

```
TreeSet<Integer> set = new TreeSet<>();

TreeSet<String> set = new TreeSet<>(
    Comparator.reverseOrder());
```

HOW TREESET WORKS

A Red-Black Tree keeps elements balanced and sorted; in-order traversal yields the sorted sequence.




```
In-order traversal (sorted order): 20, 30, 40, 50, 60, 70, 80
```

Elements must be Comparable, or a Comparator must be provided.

KEY TAKEAWAY Guarantees O(log n) time for basic operations because the tree stays balanced.

TREESET — COMMON METHODS (1 OF 2)

Basic set operations shared with every Set implementation.

METHOD	DESCRIPTION	EXAMPLE
`add(E e) / remove(Object o)`	Adds if absent / removes the element	set.add("A");
`contains(Object o)`	Checks if element exists	set.contains("A");
`size() / isEmpty()`	Count elements / check empty	set.size();
`first() / last()`	Returns lowest / highest element	set.first();

KEY TAKEAWAY The navigation methods that set TreeSet apart from HashSet continue on the next slide.

TREESET — COMMON METHODS (2 OF 2)

The navigation methods that set TreeSet apart from HashSet.

METHOD	DESCRIPTION	EXAMPLE
`ceiling(E e) / floor(E e)`	Least element ≥ e / greatest element ≤ e	set.ceiling(40);
`higher(E e) / lower(E e)`	Least element > e / greatest element < e	set.higher(40);
`subSet(from, to)`	View of elements in range	set.subSet(20, 60);
`headSet(to) / tailSet(from)`	Elements below to / at-or-above from	set.headSet(60);
`descendingSet()`	Returns a reverse-order view	set.descendingSet();

KEY TAKEAWAY ceiling/floor/higher/lower give you nearest-neighbor search — something a HashSet simply can't do.

NATURAL VS CUSTOM ORDERING

Sort by Comparable by default, or supply your own Comparator.

```
// Natural Ordering
TreeSet<String> set = new TreeSet<>();
set.add("Banana"); set.add("Apple");
set.add("Cherry");
// [Apple, Banana, Cherry]
```

```
// Custom Ordering
TreeSet<String> set = new TreeSet<>(
    Comparator.reverseOrder());
// [Cherry, Banana, Apple]
```

✅ When to Use

- Sorted order, range queries needed.
- All core operations (add/remove/contains) run in O(log n).

🚫 When Not To

- Sorted order not needed (use HashSet).

KEY TAKEAWAY TreeSet elements must be mutually comparable — a ClassCastException occurs otherwise. Shares its tree engine with TreeMap.

MAP INTERFACE

Java Collections Framework

Key Characteristics

- No duplicate keys
- Values can be duplicate
- Each key maps to one value
- Order depends on implementation
- Null keys and values allowed by some implementations

«interface»

Map<K, V>

Maps unique keys to values.
Keys cannot be duplicated;
each key maps to at most one value.

Common Operations

- V put(K key, V value)
- V get(Object key)
- V remove(Object key)
- boolean containsKey(Object key)
- boolean containsValue(Object value)
- Set<K> keySet()
- Collection<V> values()
- Set<Map.Entry<K,V>> entrySet()

HashMap<K, V>

- Hash table based
- No ordering
- Allows one null key and multiple null values
- Not synchronized

LinkedHashMap<K, V>

- Hash table + Linked list
- Maintains insertion order
- Allows one null key and multiple null values
- Not synchronized

TreeMap<K, V>

- Red-Black tree based
- Keys sorted in natural order or by Comparator
- No null keys
- Not synchronized

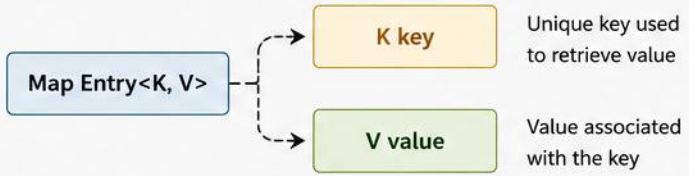
Hashtable<K, V>

- Hash table based
- Synchronized (thread-safe)
- Does not allow null keys or null values
- Legacy class

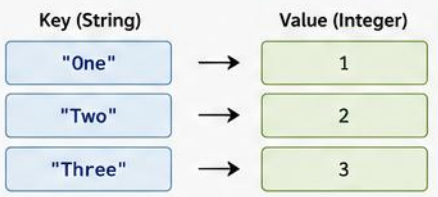
ConcurrentHashMap<K, V>

- Hash table based
- Concurrent and thread-safe
- Does not allow null keys or null values
- High concurrency

Map Entry



Example: HashMap<String, Integer>




```
Map<String, Integer> map = new HashMap<>();
map.put("One", 1);
map.put("Two", 2);
map.put("Three", 3);

int val = map.get("Two");  // returns 2
boolean hasKey = map.containsKey("One"); // true
```

Extends Implements

MAP HIERARCHY & COMMON METHODS

Five implementations with distinct ordering stories, and the core operations they all share.

`HashMap<K,V>`	`LinkedHashMap<K,V>`	`TreeMap<K,V>`	`Hashtable<K,V>`	`ConcurrentHashMap<K,V>`
Unordered	Insertion Order	Sorted Order	Legacy, Synchronized	Unordered, Concurrent

METHOD	DESCRIPTION	EXAMPLE
`put(K key, V value)`	Adds or updates the mapping	map.put(1, "A");
`get(Object key)`	Returns value for the key	map.get(1);
`remove(Object key)`	Removes the mapping	map.remove(1);
`containsKey / containsValue`	Checks if key / value exists	map.containsKey(1);
`keySet() / values() / entrySet()`	Returns keys / values / pairs	map.entrySet();

KEY TAKEAWAY entrySet() is the most efficient way to iterate both keys and values together. Map is not a Collection — it models key-value pairs.

MAP IMPLEMENTATIONS COMPARISON

Five implementations, side by side.

IMPLEMENTATION	ORDER	NULL KEY	GET/PUT	USE CASE
`HashMap`	No order	1 allowed	O(1) avg	General-purpose, best performance
`LinkedHashMap`	Insertion order	1 allowed	O(1) avg	Preserve insertion order
`TreeMap`	Sorted	Not allowed	O(log n)	Sorted keys, range operations
`Hashtable`	No order	Not allowed	O(1) avg	Legacy synchronized class
`ConcurrentHashMap`	No order	Not allowed	O(1) avg	Thread-safe, high concurrency

KEY TAKEAWAY ConcurrentHashMap, not synchronizedMap(), is the modern choice for thread-safe maps.

EXAMPLE: BASIC USAGE WITH HASHMAP

put() updates the value if the key already exists.

```
Map<Integer, String> map = new HashMap<>();
map.put(1, "Apple"); map.put(2, "Banana"); map.put(3, "Cherry");
map.put(2, "Blueberry");   // updates value for key 2

System.out.println(map.get(1));            // Apple
System.out.println(map.get(2));            // Blueberry

for (Map.Entry<Integer, String> e : map.entrySet()) {
    System.out.println(e.getKey() + " -> " + e.getValue());
}
```

KEY TAKEAWAY Map.Entry<K,V> represents a single key-value pair — that's what entrySet() iterates over.

WHEN TO USE MAP & MAP VIEWS

Three views into the same underlying data.

✅ When to Use

- Data as key-value pairs, unique keys.
- Fast retrieval by key.

🚫 When Not To

- Only need a list of values (use List).
- Frequently search by value, not key.

VIEW METHOD	RETURNS
`keySet()`	Set<K> of all keys
`values()`	Collection<V> of all values
`entrySet()`	Set<Map.Entry<K, V>> of all mappings

KEY TAKEAWAY HashMap is fastest, LinkedHashMap preserves order, TreeMap keeps keys sorted — choose accordingly.

HASHMAP<K, V>

Implementation of Map using a hash table — unique keys, one null key, multiple null values.

- Stores key-value pairs; keys must be unique, values can be duplicate.
- Not synchronized by default; provides constant-time average performance.

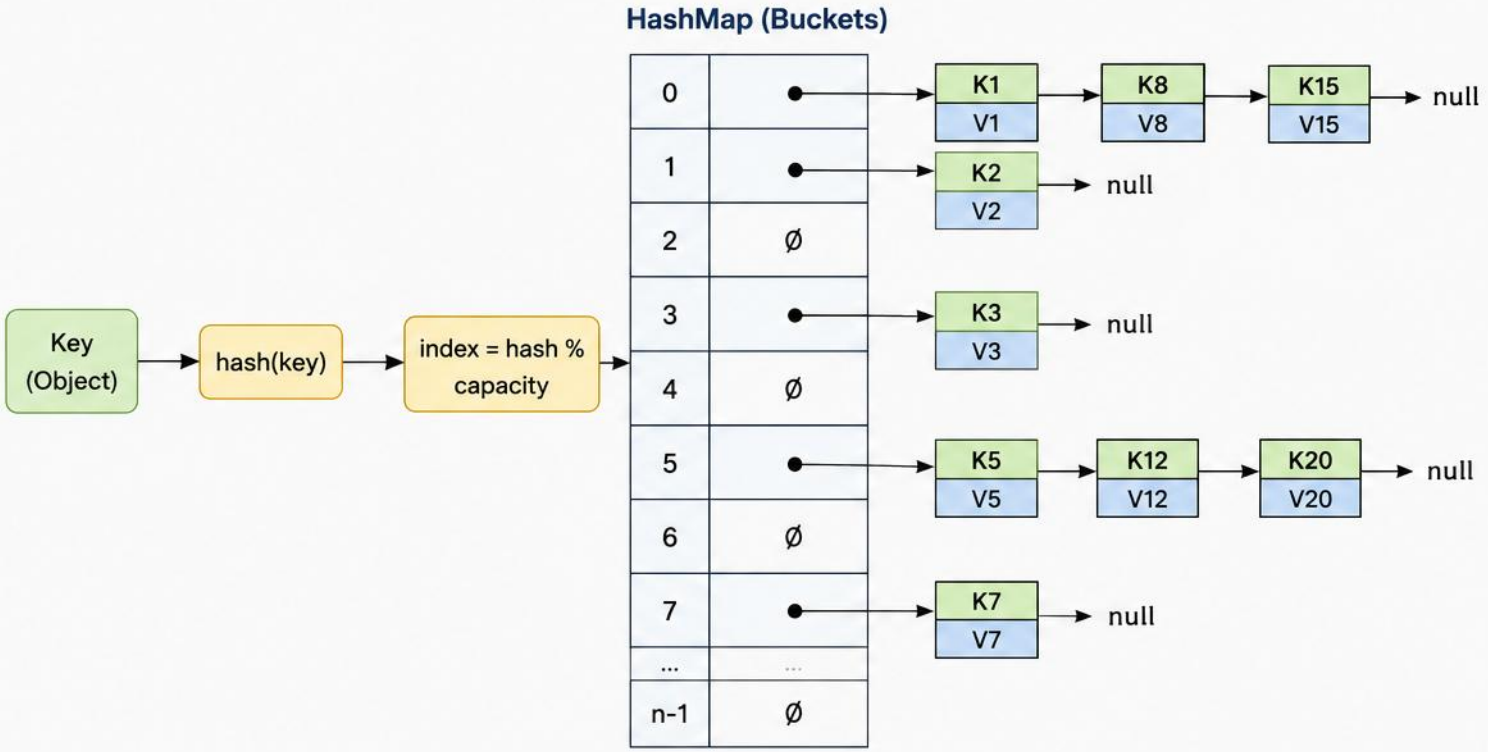
INTERNAL WORKING

- HashMap stores data in an array of buckets (Node[] table); each key is hashed to a bucket index.
- Collisions are stored in a linked list within that bucket.
- Java 8+: if bucket size ≥ 8 and table size ≥ 64, the linked list becomes a Red-Black Tree for faster lookups.

KEY TAKEAWAY HashMap is the most commonly used Map — excellent average performance, but no ordering guarantee.

HASHMAP (Java)

HashMap stores data in key-value pairs. It uses a hash function to compute an index's (bucket) for the key. If multiple keys map to the same index, they are stored in a linked list.



Example Key-Value Pairs

Key	Value
K1	V1
K2	V2
K3	V3
K5	V5
K7	V7
K8	V8
K12	V12
K15	V15
K20	V20
...	...

Key Points

- Fast operations: get(), put(), remove() – Average Time Complexity: O(1)
- Uses hashing for efficient data retrieval.
- Handles collisions using Separate Chaining (Linked Lists).
- Does not maintain any order of elements.

Common Operations

- put(key, value) – Adds or updates a key-value pair
- get(key) – Returns value for the key
- remove(key) – Removes the key-value pair
- containsKey(key) – Checks if key exists
- size() – Returns number of key-value pairs

HASHMAP — CONSTRUCTORS & DEFAULTS

Capacity and load factor together decide when a resize happens.

- HashMap() — default capacity 16, load factor 0.75.
- HashMap(int initialCapacity)
- HashMap(int initialCapacity, float loadFactor)
- HashMap(Map<? extends K, ? extends V> m) — copies all mappings.

DEFAULT CAPACITY	DEFAULT LOAD FACTOR	RESIZE TRIGGER	NEW CAPACITY
16	0.75	size > capacity × loadFactor	Typically doubled

KEY TAKEAWAY Rehashing is relatively expensive — pre-sizing a HashMap can meaningfully improve performance.

EXAMPLE: BASIC USAGE

One null key, multiple null values — both are allowed.

```
Map<Integer, String> map = new HashMap<>();
map.put(1, "Apple"); map.put(2, "Banana"); map.put(3, "Cherry");
map.put(2, "Blueberry");   // updates value for key 2
map.put(null, "No Key");  // one null key allowed
map.put(4, null);         // multiple null values allowed

System.out.println(map.get(1));                      // Apple
System.out.println(map.containsValue("Blueberry"));  // true
```

KEY TAKEAWAY When to use HashMap: fast key-based lookup where order doesn't matter — caches, indexes, dictionaries. putIfAbsent()/replace() add atomicity.

HASH FUNCTION, EQUALITY & RESIZING

Correct equals()/hashCode() implementations are essential for correct behavior.

Hash Function & Equality

- Uses key.hashCode() to find the bucket.
- Equal keys must have the same hashCode().
- Bucket index ≈ (n - 1) & hashCode.

Resizing (Rehashing)

- Occurs when size > capacity × loadFactor.
- New capacity = oldCapacity × 2.
- All entries are rehashed into the new table.

```
Map<String, Integer> map = new HashMap<>();
map.put(null, 100); map.put(null, 200);  // updates the null key
System.out.println(map.get(null));       // 200
```

KEY TAKEAWAY Only one null key is allowed — putting a second null value just updates the existing entry. Basic ops run O(1) average, O(n) worst case.

TRY IT YOURSELF — EXERCISE 3: WORKING WITH MAP

Reinforces: HashMap, as just covered.

#	STEP	CODE	RESULT
1	Goal	CRUD on a HashMap<String, Integer> (ISBN → copies)	Key-value CRUD works
2	Declare	Map<String, Integer> copies = new HashMap<>();	Empty map ready
3	Populate	copies.put("978-...", 3);	Associates a key with a value
4	Operate	copies.get("978-..."); copies.remove("978-...");	Read and delete by key
5	Iterate	for (var e : copies.entrySet()) ...	e.getKey() / e.getValue() per pair
6	Key concept	HashMap stores key-value pairs; keys are unique, values can repeat	Ties to the Map interface you just saw
7	Reference	exercises/exercise-03-hashmap.md	Full starter code and steps

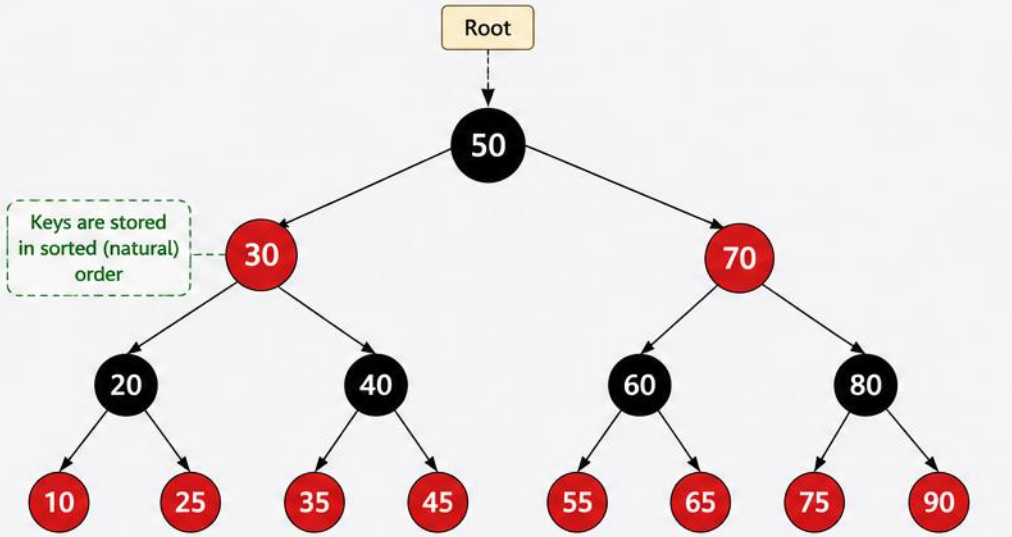
TRY IT NOW Complete Exercise 3 before continuing — see exercises/exercise-03-hashmap.md.

TreeMap<K, V> in Java

TreeMap is a Red-Black Tree based NavigableMap implementation.
It stores key-value pairs in sorted order of keys.

Key Characteristics

- Keys are unique and sorted
- Implemented using Red-Black Tree
- Provides log(n) time complexity
- Implements NavigableMap interface
- Not synchronized (use Collections.synchronizedSortedMap())
- Null key is not allowed
- Allows one null value



Tree Structure (Red-Black Tree)

- Black Node
- Red Node

Common Operations

put(K key, V value)
– Inserts a key-value pair
get(Object key)
– Retrieves value for key
remove(Object key)
– Removes key-value pair
firstKey() / lastKey()
– Returns lowest / highest key
higherKey(K key)
– Returns next higher key
lowerKey(K key)
– Returns next lower key
subMap(K from, K to)
– Returns a portion of the map

Example

```
TreeMap<Integer, String> treeMap = new TreeMap<>();
treeMap.put(50, "Fifty");
treeMap.put(30, "Thirty");
treeMap.put(70, "Seventy");
treeMap.put(20, "Twenty");
treeMap.put(40, "Forty");
...
```

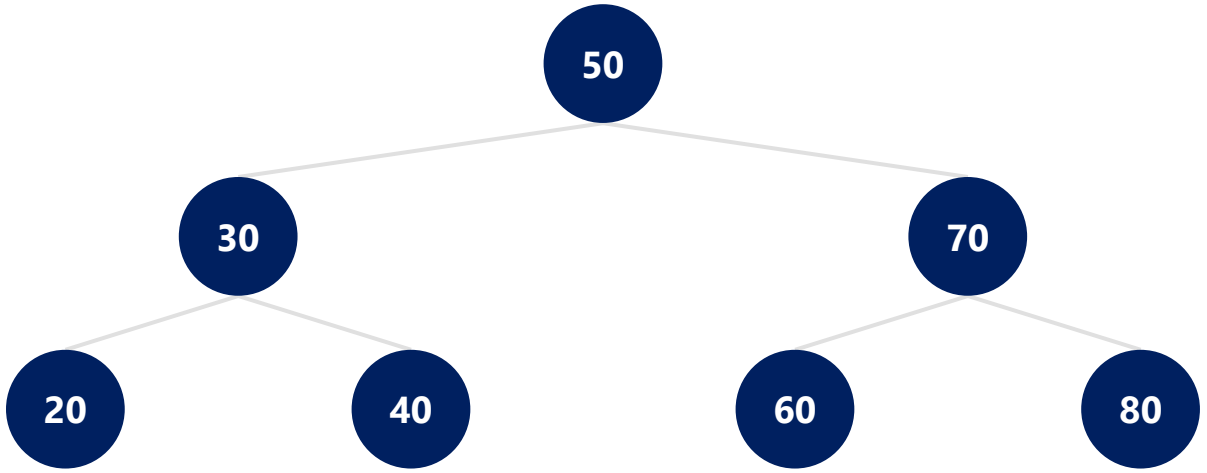
```
treeMap.get(30);         // Returns "Thirty"
treeMap.firstKey();      // Returns 10
treeMap.lastKey();       // Returns 90
treeMap.higherKey(40);   // Returns 45
treeMap.lowerKey(60);    // Returns 55
treeMap.subMap(20, 70); // Returns keys from 20 (inclusive) to 70 (exclusive)
```

Time Complexity

put()	:	O(log n)
get()	:	O(log n)
remove()	:	O(log n)
containsKey():	:	O(log n)
Traversal	:	O(n)

HOW TREEMAP WORKS

Entries live as nodes in a Red-Black Tree; in-order traversal returns them sorted by key.




```
In-order traversal (sorted by key): 20, 30, 40, 50, 60, 70, 80
```

Guarantees O(log n) time for insertions, deletions, and lookups.

KEY TAKEAWAY The Red-Black Tree keeps the map balanced, so performance stays O(log n) regardless of insertion order.

TREEMAP — COMMON METHODS (1 OF 2)

Core put/get operations shared with every Map implementation.

METHOD	DESCRIPTION	EXAMPLE
`put(K key, V value)`	Adds or updates mapping	map.put(10, "Ten");
`get / containsKey / remove`	Retrieve / check / remove by key	map.get(10);
`firstKey() / lastKey()`	Returns the lowest / highest key	map.firstKey();

KEY TAKEAWAY The range views and nearest-key search that are TreeMap's superpower continue on the next slide.

TREEMAP — COMMON METHODS (2 OF 2)

The range views and nearest-key search that are TreeMap's superpower.

METHOD	DESCRIPTION	EXAMPLE
`ceilingKey(K) / floorKey(K)`	Least key ≥ k / greatest key ≤ k	map.ceilingKey(25);
`higherKey(K) / lowerKey(K)`	Least key > k / greatest key < k	map.higherKey(25);
`subMap(from, to)`	View from fromKey (incl.) to toKey (excl.)	map.subMap(30, 70);
`headMap(to) / tailMap(from)`	Keys below to / at-or-above from	map.headMap(50);
`descendingMap()`	Returns a reverse-order view	map.descendingMap();

KEY TAKEAWAY subMap/headMap/tailMap give you cheap range queries without scanning the whole map.

TREEMAP — PERFORMANCE & EXAMPLE

Sorted natural-order keys, with O(log n) operations throughout.

OPERATION	COMPLEXITY	NOTES
`put / get / remove / containsKey`	O(log n)	Insert, update, or lookup
`firstKey / lastKey / ceilingKey / floorKey`	O(log n)	Tree navigation
`containsValue(Object)`	O(n)	Linear value search

```
TreeMap<Integer, String> map = new TreeMap<>();
map.put(50,"Fifty"); map.put(20,"Twenty"); map.put(70,"Seventy");
System.out.println(map);                    // {20=Twenty, 50=Fifty, 70=Seventy}
System.out.println(map.ceilingKey(35));     // 50
System.out.println(map.subMap(30, 70));     // {50=Fifty}
```

KEY TAKEAWAY If multiple keys compare equal (compareTo() returns 0), the new value simply replaces the old one. Shares its tree engine with TreeSet.

TRY IT YOURSELF — EXERCISE 4: SORTED COLLECTIONS

Reinforces: TreeMap and sorted key order, as just covered.

#	STEP	CODE	RESULT
1	Goal	Build a TreeMap from a HashMap	Keys now print in sorted order
2	Declare	Map<String, Double> prices = new HashMap<>();	Unsorted map, populated first
3	Sort	new TreeMap<>(prices)	Same entries, natural key order
4	Extra methods	tree.firstKey(); tree.lastKey();	TreeMap-only, not on the Map interface
5	Key concept	TreeMap key order is guaranteed	HashMap key order is not
6	Builds on	Exercise 2 new TreeSet<>(...)	Same idea, now for key-value pairs
7	Reference	exercises/exercise-04-sorted-collections.md	Full starter code and steps

TRY IT NOW Complete Exercise 4 before continuing — see exercises/exercise-04-sorted-collections.md.

MUTABILITY VS IMMUTABILITY

Mutability: state can change after creation. Immutability: state cannot change after creation.

Mutability (State Can Change)

- Object state can be modified after creation.
- Changes affect the same object (same reference).
- Generally better performance.
- Not thread-safe by default.

Immutability (State Cannot Change)

- Any "change" creates a new object.
- Inherently thread-safe.
- More reliable, easier to reason about.

KEY TAKEAWAY Mutable classes: StringBuilder, ArrayList, HashMap, Date. Immutable classes: String, Integer, LocalDate, List.of().

MUTABILITY vs IMMUTABILITY IN JAVA

Mutability means an object's state can be changed after creation.
Immutability means an object's state cannot be changed after creation.

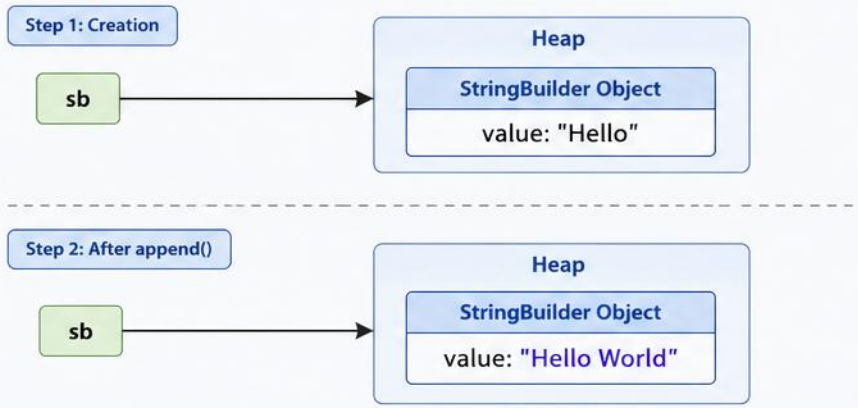
MUTABLE (Changeable)

- State can be modified after creation
- Same object reference is used
- Changes reflect to all references
- Not thread-safe by default

Example: StringBuilder

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");  // modifies the same object
System.out.println(sb); // Hello World
```

Memory Representation



One object, state changes.
All references see the updated value.

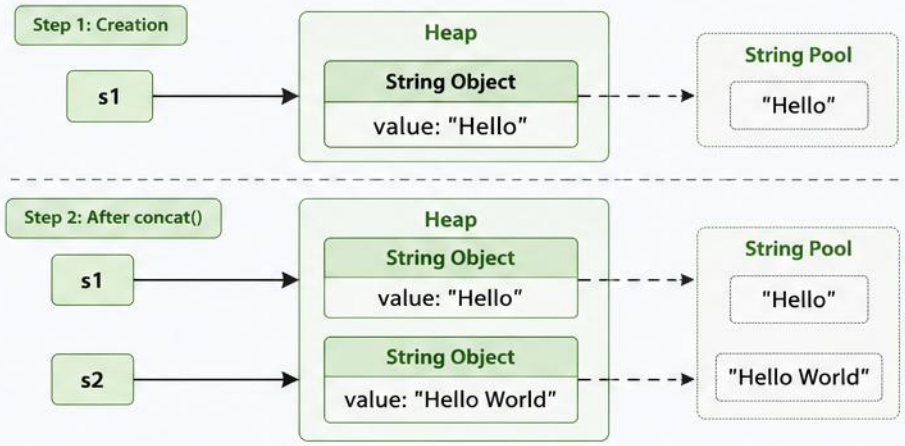
IMMUTABLE (Unchangeable)

- State cannot be modified after creation
- Any change creates a new object
- Original object remains unchanged
- Thread-safe

Example: String

```
String s1 = "Hello";
String s2 = s1.concat(" World"); // creates new object
System.out.println(s1); // Hello
System.out.println(s2); // Hello World
```

Memory Representation

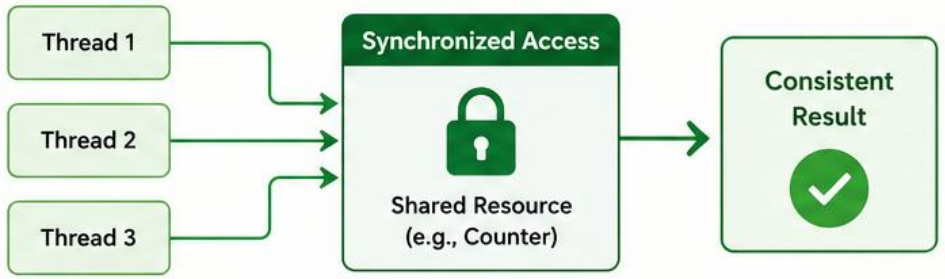


New object created for changes.
Original object remains unchanged.

Thread Safe vs Thread Unsafe in Java

THREAD SAFE

Multiple threads can access a shared resource concurrently without causing inconsistent state or data corruption.



Example (Thread Safe)

```
class Counter {
    private int count = 0;

    public synchronized void increment() {
        count++;
    }

    public synchronized int getCount() {
        return count;
    }
}
```

Why Thread Safe?

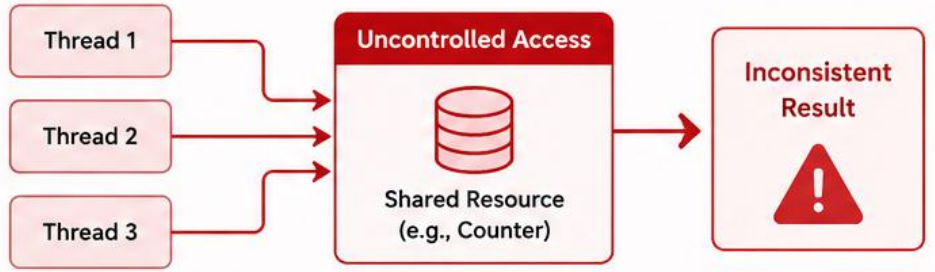
- Uses synchronization (synchronized keyword)
- Only one thread can execute critical section at a time
- Data remains consistent and accurate

Examples of Thread Safe Classes

String, Integer, Long, StringBuffer, Vector, Hashtable, ConcurrentHashMap, CopyOnWriteArrayList, etc.

THREAD UNSAFE

Multiple threads can access a shared resource concurrently and may lead to inconsistent state or data corruption.



Example (Thread Unsafe)

```
class Counter {
    private int count = 0;

    public void increment() {
        count++;
    }

    public int getCount() {
        return count;
    }
}
```

Why Thread Unsafe?

- No synchronization
- Multiple threads can modify data simultaneously
- May cause race conditions and inconsistent results

Examples of Thread Unsafe Classes

ArrayList, HashMap, HashSet, StringBuilder, SimpleDateFormat, Random, etc.

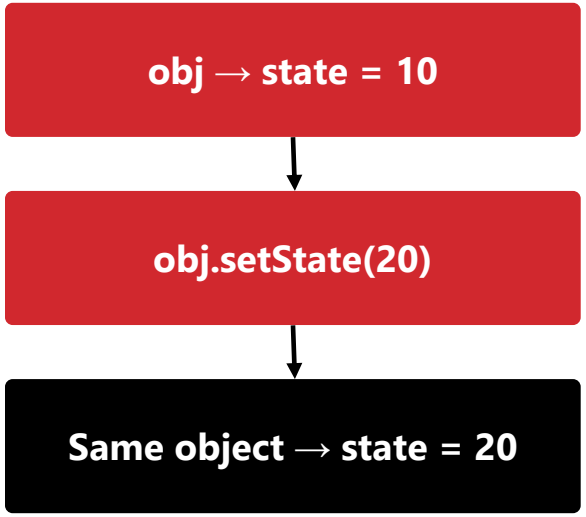
Key Takeaway

Use thread-safe classes or proper synchronization when multiple threads access shared data. This ensures data consistency, reliability, and predictable behavior in concurrent applications.

HOW THEY WORK

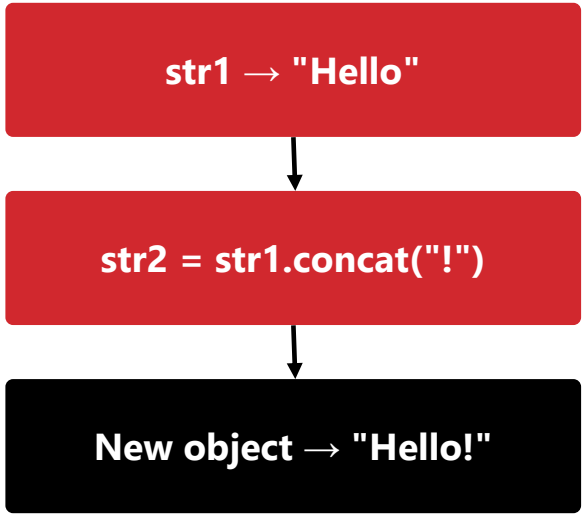
Mutation updates the same object; immutable operations return a brand-new one.

MUTABILITY



All references see the change — the object was mutated in place.

IMMUTABILITY



The original object remains unchanged — a brand-new object was returned.

KEY TAKEAWAY The same object is updated in mutability; a new object is created in immutability.

MUTABLE VS IMMUTABLE — IN CODE

StringBuilder mutates in place; String always returns something new.

MUTABLE (StringBuilder)

```
StringBuilder sb = new StringBuilder("Hello");
System.out.println(sb);   // Hello
sb.append(" World");
System.out.println(sb);   // Hello World
// Same object is modified
```

IMMUTABLE (String)

```
String s1 = "Hello";
String s2 = s1.concat(" World");
System.out.println(s1);   // Hello
System.out.println(s2);   // Hello World
// New object is created (s1 unchanged)
```

- Mutable classes: StringBuilder, StringBuffer, ArrayList, HashMap, Date, int[].
- Immutable classes: String, Integer, Long, Boolean, LocalDate, BigDecimal, List.of(), Map.of().

KEY TAKEAWAY concat() never modifies s1 — it always returns a new String, leaving the original untouched.

KEY DIFFERENCES

Mutable and immutable objects side by side.

ASPECT	MUTABLE	IMMUTABLE
`State Change`	Can be changed	Cannot be changed
`Object Creation`	Same object	New object for changes
`Thread Safety`	Not thread-safe	Thread-safe
`Performance`	Generally faster	Slightly slower (more objects)
`Memory Usage`	Less	More (new objects)
`Complexity`	More error-prone	Easier to reason about

KEY TAKEAWAY Immutability trades a little performance and memory for a lot of safety and simplicity.

WHEN TO USE WHICH & BEST PRACTICES

Default to immutability unless you have a specific reason not to.

Use Mutable When...

- Performance is critical, state changes often.
- Working with large structures needing modification.
- Memory usage must be minimized.

Use Immutable When...

- Objects are shared between threads.
- Data should remain constant (configs, keys).
- Used as HashMap/HashSet keys.

- Prefer immutability for data objects (DTOs, value objects); use final fields with no setters.
- Use immutable collections (List.of(), Map.of(), Set.of()) whenever possible.

KEY TAKEAWAY Default to immutability unless you have a specific reason to use mutability, like performance or memory constraints.

ITERATION STYLES: FOR-LOOP, ENHANCED FOR, FOREACH

Three ways to visit every element — compared concisely.

1. FOR LOOP (Arrays / Lists)

```
for (int i = 0; i < names.size(); i++) {
    String name = names.get(i);
    System.out.println(name);
}
```

- Advantages: simple, gives index access, can modify by position.
- Notes: not ideal for Set/Map; risk of IndexOutOfBoundsException.
- 2. Enhanced for-loop: for (String name : names) { ... } — clean, concise, read-only; works on arrays, collections, and maps via entrySet(). No index access.
- 5. forEach() + lambda (Java 8+): names.forEach(name -> ...) — most concise; map.forEach((k,v) -> ...) for Maps. No break/continue, no index.
- Iterating a Map: for (Map.Entry<K,V> e : map.entrySet()) is fastest when you need both key and value — keySet() alone forces a second get() per key.

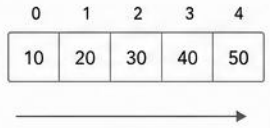
KEY TAKEAWAY The traditional for-loop is the only technique that gives you the index for free.

ITERATION TECHNIQUES IN JAVA

Different ways to traverse elements of Collections, Arrays and other data structures in Java

1 For Loop

Index-based iteration




```
int[] arr = {10, 20, 30, 40, 50};
for (int i = 0; i < arr.length; i++) {
    int value = arr[i];
    System.out.println(value);
}
```

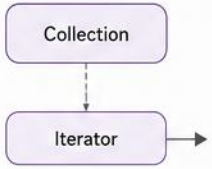
2 Enhanced For Loop (For-Each)

Simplified iteration


```
int[] arr = {10, 20, 30, 40, 50};
for (int value : arr) {
    System.out.println(value);
}
```

3 Iterator

Via Iterator interface




```
Iterator<Integer> it = list.iterator();
while (it.hasNext()) {
    Integer value = it.next();
    System.out.println(value);
}
```

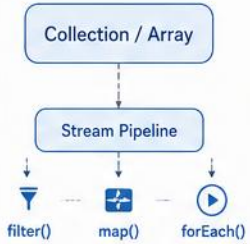
4 ListIterator

Bi-directional iteration


```
ListIterator<String> it = list.listIterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
while (it.hasPrevious()) {
    System.out.println(it.previous());
}
```

5 Stream API

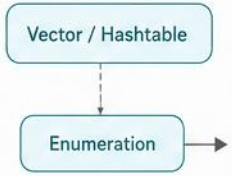
Functional style iteration




```
list.stream()
    .filter(s -> s.startsWith("A"))
    .map(String::toUpperCase)
    .forEach(System.out::println);
```

6 Enumeration

Legacy (for Vector, Hashtable)




```
Enumeration<String> e = vector.elements();
while (e.hasMoreElements()) {
    String value = e.nextElement();
    System.out.println(value);
}
```

COMPARISON AT A GLANCE

	For Loop	Enhanced For Loop	Iterator	ListIterator	Stream API	Enumeration
Use Case	Arrays, Lists (index access needed)	Collections, Arrays (simple read-only)	Collections (safe removal)	Lists (bi-directional access)	Collections, Arrays (functional operations)	Legacy Collections (Vector, Hashtable)
Direction	Forward	Forward	Forward	Forward & Backward	Forward (pipeline)	Forward
Remove Support	No	No	Yes (it.remove())	Yes (it.remove())	No (without collect) (Use collect to modify)	No
Performance	Fast	Fast	Moderate	Moderate	Depends on operations	Slow (Legacy)
Introduced In	Java 1.0	Java 1.5	Java 1.2	Java 1.2	Java 8	Java 1.0

BEST PRACTICES

- Use Enhanced For Loop for simple read-only traversal.
- Use Iterator when you need to remove elements while iterating.
- Use ListIterator for bi-directional traversal of Lists.
- Use Stream API for declarative, functional and parallel processing.
- Avoid Enumeration in modern applications (legacy).

3. ITERATOR & 4. LISTITERATOR

Explicit iteration objects — one for safe removal, one for bidirectional traversal.

3. ITERATOR — SAFE REMOVAL

```
Iterator<String> it = names.iterator();
while (it.hasNext()) {
    String name = it.next();
    if (name.equals("B")) {
        it.remove();   // safe removal
    }
}
```

4. LISTITERATOR — BIDIRECTIONAL

```
ListIterator<String> li = names.listIterator();
while (li.hasNext()) {   // forward
    System.out.println(li.next());
}
while (li.hasPrevious()) {   // backward
    System.out.println(li.previous());
}
```

- Iterator works for all Collection implementations; ListIterator is List-only but can add, set, and remove in both directions. Modifying a collection any other way while iterating (e.g. list.remove() inside a for-each) throws ConcurrentModificationException — Iterator.remove() (or ListIterator's add/set/remove) is the only safe way. Only the plain for-loop and ListIterator give index/positional access; only Iterator and ListIterator support safe removal.

KEY TAKEAWAY Iterator.remove() is the only safe way to remove elements while iterating.

TRY IT YOURSELF — EXERCISE 5: SAFE ITERATION

Reinforces: the iteration techniques you just saw.

#	STEP	CODE	RESULT
1	Goal	Use Iterator.remove() to delete elements mid-loop safely	No ConcurrentModificationException
2	Declare	Iterator<String> it = list.iterator();	Positioned before the first element
3	Loop	while (it.hasNext()) { String s = it.next(); ... }	Advances one element per call
4	Remove	if (condition) it.remove();	Safely removes the current element
5	Avoid	list.remove(s) inside a for-each loop	Throws ConcurrentModificationException
6	Key concept	Only the Iterator itself may safely modify a collection mid-loop	Ties to the iteration techniques you just saw
7	Reference	exercises/exercise-05-iteration.md	Full starter code and steps

TRY IT NOW Complete Exercise 5 before continuing — see exercises/exercise-05-iteration.md.

JAVA COLLECTION PERFORMANCE COMPARISON

Average time complexity of common operations, at a glance.

Excellent

O(1)

Good

O(log n) or amortized O(1)

Poor

O(n)

- Hash-based collections (HashMap, HashSet) offer the best average performance for most operations.
- Tree-based collections (TreeMap, TreeSet) provide sorted order with O(log n) operations.
- ArrayList is ideal for random access; LinkedList for frequent insertions/deletions in the middle.
- Choose based on your use case: need order, need speed, or need both.

KEY TAKEAWAY These are average-case complexities — worst case for hash-based collections can degrade to O(n).

JAVA COLLECTION PERFORMANCE COMPARISON

Collection Type	Access (get) Average Case	Insert (add) Average Case	Delete (remove) Average Case	Search (contains) Average Case	Iteration (for-each) Average Case	Key Characteristics
ArrayList	O(1)	O(1)*	O(n)	O(n)	O(n)	• Dynamic array • Fast access, slow insert/delete in middle
LinkedList	O(n)	O(1)**	O(1)**	O(n)	O(n)	• Doubly linked list • Fast insert/delete, slow access
HashSet	N/A	O(1)	O(1)	O(1)	O(n)	• Hash table • Unordered, no duplicates
LinkedHashSet	N/A	O(1)	O(1)	O(1)	O(n)	• Hash table + Linked list • Maintains insertion order
TreeSet	N/A	O(log n)	O(log n)	O(log n)	O(n)	• Red-Black Tree • Sorted order, no duplicates
HashMap	O(1)	O(1)	O(1)	O(1)	O(n)	• Hash table • Unordered key-value pairs
LinkedHashMap	O(1)	O(1)	O(1)	O(1)	O(n)	• Hash table + Linked list • Maintains insertion order
TreeMap	O(log n)	O(log n)	O(log n)	O(log n)	O(n)	• Red-Black Tree • Sorted by keys

Legend: Excellent (O(1)) | Good (O(log n)) | Poor (O(n)) | N/A Not Applicable

* Amortized O(1). Resize is O(n)
** When adding/removing at head or known position

UNDERLYING DATA STRUCTURES & BIG-O CHEAT SHEET

What's really happening under the hood.

COLLECTION	DATA STRUCTURE
`ArrayList`	Resizable Array
`LinkedList`	Doubly Linked List
`HashSet`	Hash Table (backed by HashMap)
`TreeSet`	Red-Black Tree (backed by TreeMap)
`HashMap`	Hash Table
`TreeMap`	Red-Black Tree

COMPLEXITY	MEANING
`O(1)`	Constant — independent of input size (best)
`O(log n)`	Logarithmic — grows slowly with n
`O(n)`	Linear — grows directly with n (worst here)

KEY TAKEAWAY Every performance characteristic in this section traces back to one of these six data structures.

CHOOSING THE RIGHT COLLECTION

A practical guide to select the best Collection or Map for your use case.

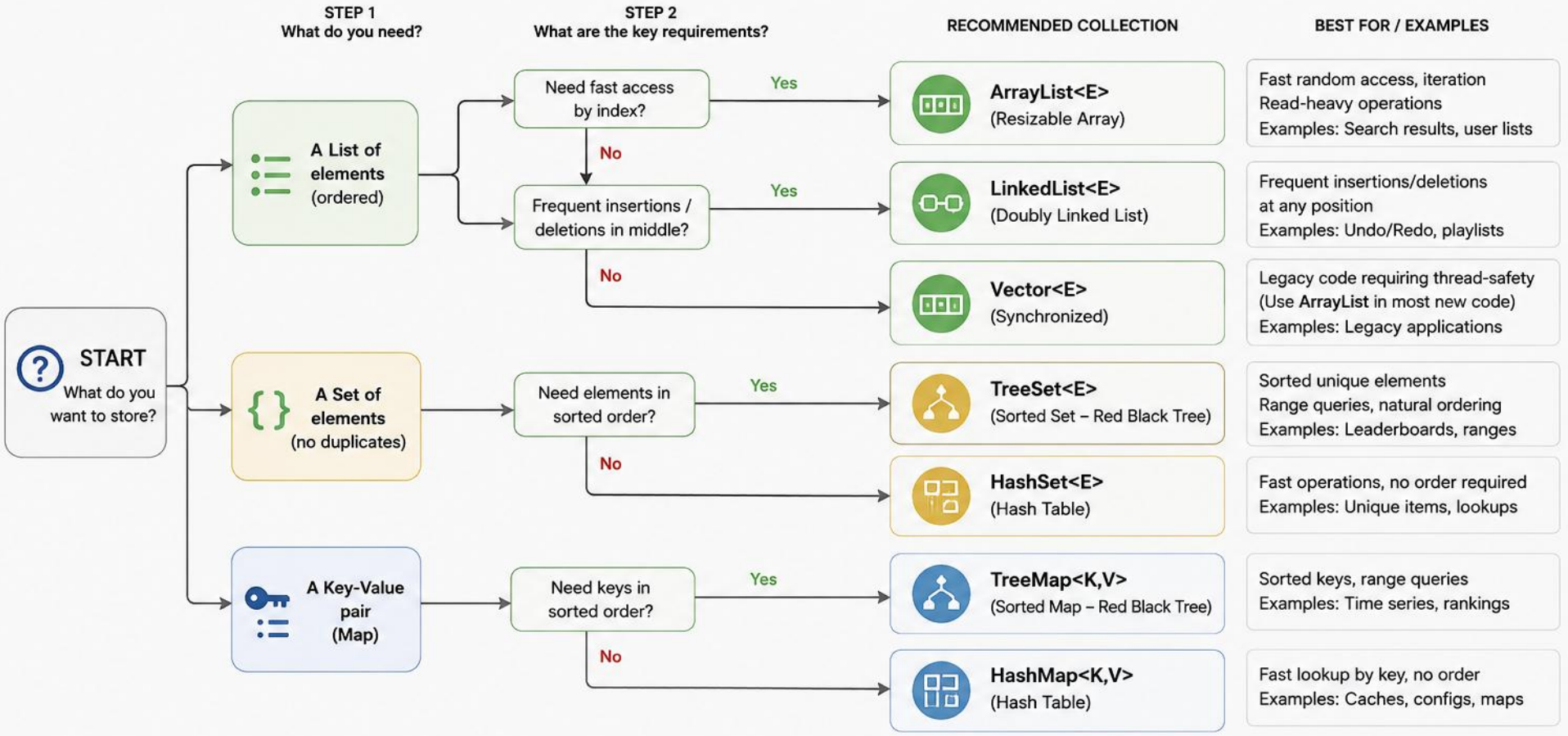
BEFORE CHOOSING, ASK YOURSELF:

- Do you need to store individual objects, or key-value pairs?
- Should duplicates be allowed?
- Do you need elements in a specific order?
- How important is performance, and where are most operations happening — start, middle, or end?
- Do you need access by index?
- Is the collection read-heavy or write-heavy?
- Will the collection be accessed by multiple threads?

KEY TAKEAWAY The right choice makes your code faster and cleaner — always start with these seven questions.

CHOOSING THE RIGHT COLLECTION IN JAVA

Select the most appropriate Collection based on your data needs



QUICK REFERENCE

ArrayList	LinkedList	Vector	HashSet	TreeSet	HashMap	TreeMap
✓ Fast random access	✓ Fast insert/delete	✓ Synchronized	✓ No duplicates	✓ No duplicates	✓ Key-Value pairs	✓ Key-Value pairs
✓ Resizable array	✓ Doubly linked list	✓ Legacy class	✓ No order guarantee	✓ Sorted order	✓ No order guarantee	✓ Sorted by keys
✓ Not synchronized	✓ More memory	✓ Use ArrayList instead	✓ Backed by HashMap	✓ Slower than HashSet	✓ Allows one null key	✓ Slower than HashMap

Legend
- List
- Set
- Map

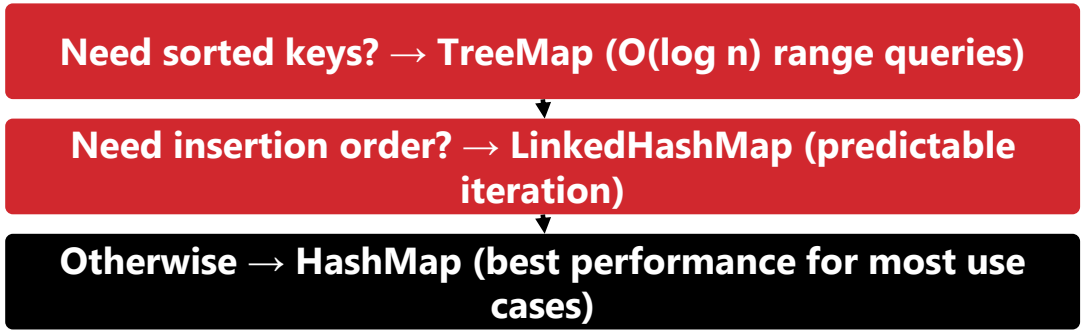
General Rule: Prefer **ArrayList**, **HashSet**, **HashMap** for most use cases unless you have a specific reason (sorting, frequent middle updates, legacy, etc.).

DECISION FLOW — COLLECTIONS & MAPS

Sorted? Unique? Duplicates? A few questions narrow it down fast, for both.

A. INDIVIDUAL OBJECTS (COLLECTION)

B. KEY-VALUE PAIRS (MAP)



KEY TAKEAWAY When in doubt, ArrayList and HashMap are almost always the right defaults.

COLLECTION / MAP COMPARISON — ORDER & NULLS (1 OF 2)

Order, duplicates, and nulls for the List and Set implementations.

TYPE	ORDER	DUPLICATES	NULLS
`ArrayList`	Insertion order	Allowed	Yes (multiple)
`LinkedList`	Insertion order	Allowed	Yes (multiple)
`HashSet`	No order	Not allowed	Yes (1 null)
`LinkedHashSet`	Insertion order	Not allowed	Yes (1 null)
`TreeSet`	Sorted	Not allowed	No

KEY TAKEAWAY TreeSet is the strictest of the five — sorted, no duplicates, and no nulls at all.

COLLECTION / MAP COMPARISON — ORDER & NULLS (2 OF 2)

Order, duplicates, and nulls for the Map implementations.

TYPE	ORDER	DUPLICATES	NULLS
`HashMap`	No order	Keys: no, Values: yes	1 null key, many null values
`LinkedHashMap`	Insertion order	Keys: no, Values: yes	1 null key, many null values
`TreeMap`	Sorted by key	Keys: no, Values: yes	No null keys

KEY TAKEAWAY Nulls are the detail everyone forgets — TreeSet and TreeMap reject them outright.

COLLECTION / MAP COMPARISON — TIME COMPLEXITY (1 OF 2)

Add, remove, get, and search costs for the List and Set implementations.

TYPE	ADD	REMOVE	GET	SEARCH
`ArrayList`	Amortized O(1)	O(n)	O(1)	O(n)
`LinkedList`	O(1)	O(1)	O(n)	O(n)
`HashSet`	O(1)	O(1)	—	O(1)
`LinkedHashSet`	O(1)	O(1)	—	O(1)
`TreeSet`	O(log n)	O(log n)	—	O(log n)

KEY TAKEAWAY TreeSet trades O(1) speed for guaranteed sorted order — O(log n) is the price of staying sorted.

COLLECTION / MAP COMPARISON — TIME COMPLEXITY (2 OF 2)

Add, remove, get, and search costs for the Map implementations.

TYPE	ADD	REMOVE	GET	SEARCH
`HashMap`	O(1)	O(1)	O(1)	O(1)
`LinkedHashMap`	O(1)	O(1)	O(1)	O(1)
`TreeMap`	O(log n)	O(log n)	O(log n)	O(log n)

KEY TAKEAWAY The Linked* variants match their non-linked counterparts in speed — the only cost is a little extra memory.

ONE-LINERS & BEST PRACTICES

Declaring each collection, and the habits that keep code clean.

```
List<String> list = new ArrayList<>();
List<String> linked = new LinkedList<>();

Set<String> set = new HashSet<>();
Set<String> tree = new TreeSet<>();

Map<String, Integer> map = new HashMap<>();
Map<String, Integer> tm = new TreeMap<>();
```

- Program to interfaces (List, Set, Map), not implementations.
- Choose the simplest collection that meets your requirements.
- Consider ordering, uniqueness, and performance before choosing.
- Use an appropriate initial capacity for better performance.
- Avoid unnecessary synchronization; use concurrent collections when needed.
- Profile your application — real data may differ from theoretical complexity.

KEY TAKEAWAY There is no one-size-fits-all solution — the right collection depends on your data and operations.

KEY TAKEAWAY

Data type, ordering, uniqueness, performance — in that order.

Understand your requirements first — data type, ordering, uniqueness, and performance — then choose the collection that best fits your use case.

TIP: There is no one-size-fits-all solution.

- The right choice depends on your data model.
- The operations you need to perform.
- Your performance requirements.

KEY TAKEAWAY The "right" collection depends on your data, required operations, and performance requirements.

TRY IT YOURSELF — EXERCISE 6: CHOOSE THE RIGHT COLLECTION

Reinforces: the collection comparison you just saw.

#	STEP	CODE	RESULT
1	Goal	Justify List vs Set vs Map for ordered tasks, unique tags, id→user	Choices match List/Set/Map roles
2	Ordered tasks	List (e.g. ArrayList)	Preserves insertion order, allows duplicates
3	Unique tags	Set (e.g. HashSet)	Automatically rejects duplicate values
4	id → user lookup	Map (e.g. HashMap)	Fast lookup by a unique key
5	Deliverable	A short List / Set / Map choice table with your reasoning	No code required
6	Key concept	Match the collection to the shape of the problem, not habit	Ties to the comparison tables you just saw
7	Reference	exercises/exercise-06-choose-collection.md	Full starter code and steps

TRY IT NOW Complete Exercise 6 before continuing — see exercises/exercise-06-choose-collection.md.

ENTERPRISE COLLECTION USAGE PATTERNS

How and when to use Java Collections and Maps in real enterprise applications.

- Choose the right collection based on data characteristics and business needs.
- Consider ordering, uniqueness, performance, and access patterns.
- Design for scalability, thread safety, and maintainability.
- Follow proven enterprise usage patterns; measure and profile real performance.

- Usage Principles: model data and access patterns before choosing; prefer interface types in APIs.
- Use the simplest collection that satisfies requirements; use concurrent collections in multi-threaded apps.

KEY TAKEAWAY Enterprise-scale decisions are the same fundamentals, applied consistently across a whole codebase.

ENTERPRISE COLLECTION USAGE PATTERNS (Java)

Choose the right Collection for common enterprise use cases to ensure performance, scalability and maintainability.

USE CASE / PATTERN	RECOMMENDED COLLECTION	KEY CHARACTERISTICS	TYPICAL ENTERPRISE SCENARIOS
Ordered Sequence Maintain insertion order, allow duplicates, index access	ArrayList<E>	• Dynamic array • Fast random access O(1) • Fast iteration • Add/remove at end amortized O(1)	• Caching rows • Search results • Configuration lists • Pagination data
Frequent Insert/Delete Frequent additions or removals in middle	LinkedList<E>	• Doubly linked list • Fast insert/delete O(1) • No random access O(n) • Higher memory overhead	• Task queues • Undo/Redo operations • Message processing
Unique Elements / Membership Check No duplicates, fast lookup	HashSet<E>	• Hash table based • Fast add/remove/contains O(1) • No order guaranteed	• User roles/permissions • Feature flags • De-duplication • Access control lists
Sorted Unique Elements No duplicates, sorted order by natural/comparator	TreeSet<E>	• Red-Black tree based • Elements sorted • Add/remove/contains O(log n) • Navigable (headSet, tailSet, etc.)	• Leaderboards • Range queries • Sorted reports • Scheduling/time slots
Key-Value Lookup Fast lookup by key, unique keys	HashMap<K,V>	• Hash table based • Fast put/get/remove O(1) • No order guaranteed • Keys must be unique	• Caches • Session stores • Configuration maps • Reference data
Sorted Key-Value Keys sorted, range queries on keys	TreeMap<K,V>	• Red-Black tree based • Keys sorted • put/get/remove O(log n) • Navigable (subMap, headMap, etc.)	• Time series data • Financial data • Range lookups • Scheduling

Quick Guide

- Need order & index? → Use ArrayList
- Frequent insert/delete? → Use LinkedList
- Need uniqueness & fast lookup? → Use HashSet
- Need sorted unique set? → Use TreeSet
- Key-Value lookup? → Use HashMap
- Sorted keys & range queries? → Use TreeMap

Best Practice

- Program to interfaces (List, Set, Map)
- Choose based on use case, not habit
- Consider performance & memory trade-offs

ENTERPRISE PATTERNS

Seven recurring patterns and the collections that implement them.

1. Caching

- LinkedHashMap (accessOrder), ConcurrentHashMap
- Use: LRU caches, session stores

2. Lookup / Indexing

- HashMap, ConcurrentHashMap
- Use: ID lookup, SKU lookup, config

3. Unique Items

- HashSet, LinkedHashSet (order matters)
- Use: roles, tag dedup, permissions

4. Sorted Data

- TreeSet, TreeMap
- Use: leaderboards, time-series

5. Ordered Data

- ArrayList (random access), LinkedList (insert/delete)
- Use: order history, task queues

6. Key-Value Config

- HashMap, Properties (String→String)
- Use: settings, feature flags

7. Grouping / Multimap

- Map<Key, List<Value>>, Map<Key, Set<Value>>
- Use: orders by customer, events by date

KEY TAKEAWAY Caching and lookup are the two most common enterprise patterns — both usually start with a Map.

ANTI-PATTERNS TO AVOID & BEST PRACTICES

What experienced teams stop doing, and what they do instead.

Anti-Patterns to Avoid

- Using Vector or Hashtable in new code.
- Using ArrayList when only uniqueness is needed (use Set).
- Using LinkedList for random access (use ArrayList).
- Using TreeSet/TreeMap when sorting isn't required.
- Exposing concrete collection types in public APIs.

Best Practices

- Code to interfaces, not implementations.
- Use generics to ensure type safety.
- Set appropriate initial capacity to reduce resizing.
- Use immutable collections for read-only data.
- Use the Streams API for declarative data processing.

KEY TAKEAWAY Ignoring initial capacity is one of the quietest performance bugs — it causes silent resizing overhead.

SUMMARY

Understand your data, access patterns, and business requirements before selecting a collection.

Choosing the right Collection or Map helps build applications that are scalable, maintainable, and high-performance.

TIP: There is no single "best" collection. The best choice depends on:

- Your data model and required operations.
- Ordering requirements and concurrency needs.
- Performance characteristics.

KEY TAKEAWAY There is no single "best" collection — the best choice depends on your data model, operations, ordering, and concurrency needs.

TRY IT YOURSELF — EXERCISE 7: LIBRARY WARM-UP

Reinforces: using List and Map together, before the graded Lab 5 project.

#	STEP	CODE	RESULT
1	Goal	Tiny service: a List of titles plus a Map of memberId to borrowed title	Checkout updates both structures
2	Declare	List<String> titles = ...; Map<String, String> borrowed = ...;	Two collections working together
3	Checkout	if (titles.remove(title)) borrowed.put(memberId, title);	Moves a title from the list into the map
4	Verify	Print titles and borrowed after checkout	titles shrinks; borrowed gains an entry
5	Key concept	Real systems combine multiple collections, like Lab 5 does	Library Management System uses this exact pattern
6	Next	Lab 5: Library Management System	The full graded project — see the following slides
7	Reference	exercises/exercise-07-library-warmup.md	Full starter code and steps

TRY IT NOW Complete Exercise 7 before continuing — see exercises/exercise-07-library-warmup.md.

LAB 5 OVERVIEW — LIBRARY MANAGEMENT SYSTEM

Build a console Library Management System using Lists, Sets, and Maps for real data.

LAB GOAL
Apply List, Set, and Map together in one working application

ESTIMATED TIME
45 Minutes

STRUCTURE
16 Guided Steps + Bonus Challenges

PREREQUISITES
Lab 0 (JDK 21), Core Java (OOP, Generics)

SKILLS YOU WILL PRACTICE

- Choosing List, Set, or Map for each domain concept
- CRUD, search, and borrow/return operations
- Comparable and Comparator sorting
- TreeSet/TreeMap reporting and insights
- Environment: JDK 21, IntelliJ IDEA (or VS Code), plain javac/java — no Maven/Spring/database required.

KEY TAKEAWAY Lab 5 turns Collections theory into a working system you compile and run yourself.

LAB 5 SUBMISSION CHECKLIST

Everything to submit, verify, and double-check before turning in the Library Management System.

📦 Deliverables

- Source code for every class
- Menu + sample-session screenshots
- Field-to-collection mapping notes
- LMS write-up with run commands
- Source under examples/Lab5-LibraryManagement/src/, notes under notes/.

🎯 Success Criteria

- Menu compiles and runs via javac -d out / java -cp out
- Duplicate book/member IDs rejected by HashSet before insert
- Reports and TreeSet/TreeMap category insights match the sample session
- All 7 reflection questions answered in notes/lab5-answers.md.

🚀 Challenge Yourself

- Add borrow history + Top 5 Borrowed Books (menu 15-16)
- Export a report to library-report.txt (menu 17)
- Add partial-title search and multi-field sort
- Time ArrayList vs LinkedList inserts (menu 14, bonus)

KEY TAKEAWAY The rubric scores collection choice and a working menu — not clever one-liners.

LAB 5 BUSINESS SCENARIO & CORE CLASSES

A campus Library Management System — the vehicle for practicing List, Set, and Map together.

What You Will Build

- Book, Member, BorrowRecord — domain models
- LibraryService — collections + operations
- ReportService — summary reporting
- BookComparator — price-based sorting

Core Collections Used

- ArrayList<Book>, ArrayList<Member> — catalog and roster
- HashSet<String> — bookIds and memberIds
- HashMap<String,String> — borrow records (bookId to memberId)
- TreeSet/TreeMap — categories and category counts

Time & Structure

- Estimated Time: 45 Minutes
- 16 Guided Steps + Bonus Challenges

KEY TAKEAWAY Every field in the Library domain maps to a deliberate List, Set, or Map choice.

LAB 5 IMPLEMENTATION STEPS (1 OF 2)

Project setup, domain models, and wiring the collections.

#	TITLE	DESCRIPTION	KEY COLLECTIONS
1	Project Setup	Create src/com/academy/library folders matching the package declaration	—
2-4	Domain Models	Book (Comparable by title), Member, and BorrowRecord with final IDs and clear toString()	—
5	Wire LibraryService Collections	Declare the List/Set/Map fields that back every catalog, ID guard, and lookup	ArrayList, HashSet, HashMap, TreeSet, TreeMap
6	Add Book & Register Member	Reject duplicate IDs via HashSet before inserting into the ArrayList catalog	ArrayList, HashSet

KEY TAKEAWAY Steps 1-6 build the domain models and wire every collection before any menu exists.

LAB 5 IMPLEMENTATION STEPS (2 OF 2)

Display, search, borrow/return, sorting, reports, and the menu.

#	TITLE	DESCRIPTION	KEY COLLECTIONS
7-8	Display & Search	Four iteration styles (for, for-each, Iterator, forEach) plus search by ID, title, author, category	Iterator, forEach
9-11	Borrow/Return, Sort & Reports	HashMap tracks borrowers; sort by title or price; reports show TreeSet/TreeMap insights	HashMap, TreeSet, TreeMap
12-16	Menu, Compile & Verify	Build the Main menu, compile with javac -d out, run the sample session, then time ArrayList vs LinkedList	ArrayList vs LinkedList

KEY TAKEAWAY Step 15's ArrayList vs LinkedList timing turns Big-O theory into numbers you measured yourself.

LAB 5 CHECKPOINTS & EVALUATION RUBRIC

Self-check before submitting Lab 5, and how the 100 points break down.

CHECKPOINT	DETAILS
`Packages & Models`	Every file declares package com.academy.library; Book, Member, (BorrowRecord) present
`Collections Wired`	List/Set/Map/TreeSet/TreeMap fields present; duplicate IDs rejected
`Borrow / Return`	HashMap tracks the current borrower; return clears the entry
`Compile & Menu`	javac -d out succeeds; java -cp out com.academy.library.Main shows the menu
`Evidence & Reflection`	Screenshots plus notes/lab5-answers.md answering the 7 reflection questions

CRITERIA	PTS
`Collections Used`	20
`Models & CRUD`	25
`Borrow/Return`	15
`Sorting`	10
`Reports`	10
`Menu & Quality`	20

KEY TAKEAWAY Correct List/Set/Map usage plus a working borrow/return flow account for well over half the score.

MODULE 5 SUMMARY

Java Collections provide powerful, ready-to-use data structures for clean, efficient, scalable applications.

The right collection, used the right way, makes your code simpler and your system faster.

Lists & Sets

- Lists — ordered, allow duplicates: ArrayList, LinkedList
- Sets — no duplicates: HashSet, LinkedHashSet, TreeSet
- Real-world use: course registration, e-commerce carts/catalogs, social feeds.

Maps & Iteration

- Maps — key-value pairs: HashMap, LinkedHashMap, TreeMap
- Iteration techniques and performance comparison
- Real-world use: inventory/config lookups, caching, reporting aggregation.

KEY TAKEAWAY Mastering the Collections Framework helps you handle data efficiently and write cleaner code.

COLLECTION QUICK GUIDE (1 OF 2)

Order, duplicates, and best-for, for the List and Set implementations.

COLLECTION	ORDER	DUPLICATES	BEST FOR
`ArrayList`	Ordered	Allowed	Fast access, frequent reads
`LinkedList`	Ordered	Allowed	Frequent insert/delete in middle
`HashSet`	No Order	Not Allowed	Fast lookups, uniqueness
`LinkedHashSet`	Insertion Order	Not Allowed	Predictable order + uniqueness
`TreeSet`	Sorted	Not Allowed	Sorted unique elements

KEY TAKEAWAY The three Map implementations continue on the next slide.

COLLECTION QUICK GUIDE (2 OF 2)

Order, duplicates, and best-for, for the Map implementations.

COLLECTION	ORDER	DUPLICATES	BEST FOR
`HashMap`	No Order	N/A	Fast key lookup
`LinkedHashMap`	Insertion Order	N/A	Predictable order of keys
`TreeMap`	Sorted by Key	N/A	Sorted keys, range queries

KEY TAKEAWAY This table is the single most useful reference from the entire module — bookmark it.

COLLECTION DECISION CHECKLIST

Performance trade-offs, best practices, and pitfalls to check before you commit to a collection.

📋 ArrayList / LinkedList

- ArrayList: fast random access (get); slower insert/delete in middle.
- LinkedList: fast insert/delete in middle; slower random access.
- Avoid: Vector or Hashtable in new code; LinkedList for random access (use ArrayList).

🔷 HashSet / HashMap

- Average O(1) for add, remove, contains, get (HashMap).
- Best practice: program to interfaces (List/Set/Map), not concrete implementations.

🌳 TreeSet / TreeMap

- O(log n) operations; keeps elements/keys sorted.
- Best practice: choose the simplest collection that meets your requirements; set an initial capacity for large collections.

KEY TAKEAWAY LinkedHashSet/LinkedHashMap trade a little speed for insertion order. Profile before optimizing — and keep exploring Concurrent Collections and the Stream API.

WHEN TO USE WHAT? (1 OF 2)

Five List and Set questions, and their answers.

NEED	USE
Frequent reads by index?	ArrayList
Frequent insertions/deletions in the middle?	LinkedList
Unique items with fast lookup?	HashSet
Uniqueness with predictable insertion order?	LinkedHashSet
Sorted unique items?	TreeSet

KEY TAKEAWAY Three Map questions continue on the next slide.

WHEN TO USE WHAT? (2 OF 2)

Three Map questions, and their answers.

NEED	USE
`Key-value pairs with fast lookup?`	HashMap
`Key insertion order?`	LinkedHashMap
`Sorted keys or range queries?`	TreeMap

KEY TAKEAWAY This decision table is worth memorizing — it's the fastest path to the right collection under time pressure.

KNOWLEDGE CHECK — MULTIPLE CHOICE (1–6)

Choose the best answer. Correct answers are marked ✓.

1. Which collection maintains insertion order and allows duplicates?
A) HashSet **B) LinkedList ✓** C) TreeSet D) LinkedHashSet

2. Which collection does NOT allow duplicate elements?
A) ArrayList **B) HashSet ✓** C) LinkedHashSet D) TreeMap

3. Which Map implementation keeps entries sorted by key?
A) HashMap B) LinkedHashMap **C) TreeMap ✓** D) Hashtable

4. What is the average time complexity of get() in HashMap?
A) O(1) ✓ B) O(log n) C) O(n) D) O(n log n)

5. Which of the following is true about ArrayList?
A) Fast inserts/deletes in the middle **B) Fast random access by index ✓** C) Maintains elements in sorted order D) Thread-safe by default

6. Best suited for unique items with frequent add, remove, and contains?
A) ArrayList **B) HashSet ✓** C) LinkedList D) TreeMap

KEY TAKEAWAY Six more multiple-choice questions continue on the next slide.

KNOWLEDGE CHECK — MULTIPLE CHOICE (7–12)

Choose the best answer. Correct answers are marked ✓.

7. Which of these would you use when you only want to read data, not modify it?
A) Collection B) Iterable C) List **D) Collections.unmodifiableList() ✓**

8. Which iterator type allows you to remove elements safely during iteration?
A) Iterator B) ListIterator C) Enhanced for-loop **D) Both A and B ✓**

9. Modifying an ArrayList while iterating (without iterator.remove()) causes:
A) The element is skipped B) The list is re-sorted **C) A ConcurrentModificationException ✓** D) The iterator resets automatically

10. Which maintains insertion order with average O(1) get/put/remove?
A) HashMap **B) LinkedHashMap ✓** C) TreeMap D) Hashtable

11. Which would you choose to maintain a sorted set of unique integers?
A) HashSet **B) TreeSet ✓** C) LinkedHashSet D) PriorityQueue

12. Which statement about HashMap is TRUE?
A) It is synchronized by default B) It does not allow null keys or values **C) It does not guarantee any order ✓** D) It supports duplicate keys

KEY TAKEAWAY 12 questions, 1 point each — that's the multiple-choice portion of this knowledge check.

MINI CHALLENGES

Three quick code-reading exercises — predict the output before checking the answer.

1 Predict the Output

- list.add("A"); list.add("B"); list.add("C");
- list.remove(1);
- System.out.println(list);
- Answer: [A, C]

2 What's the Size?

- set.add(10); set.add(20);
- set.add(10); set.add(30);
- System.out.println(set.size());
- Answer: 3 (HashSet ignores duplicates)

3 Identify the Collection

- "I maintain insertion order, allow duplicates, provide fast random access, and am backed by a dynamic array."
- Answer: ArrayList

KEY TAKEAWAY If you can answer all three without running the code, you've internalized this module.

Nicely done

You can choose the right List, Set, or Map for any job — and explain why it's the right one.

Streams and functional programming are next in your toolkit.